mathematics

MDPI

Article

PA-CFL: Privacy-Preserving Adaptive Clustered Federated Learning Under Heterogeneous Data

Taihao Zhang [1], Wenting Shen [1,*], Hanlin Zhang [1] and Chenbin Zhao [2]

[1] College of Computer Science and Technology, Qingdao University, Qingdao 266071, China; zhangtaihao@qdu.edu.cn (T.Z.); hanlin@qdu.edu.cn (H.Z.)
[2] School of Mathematics and Big Data, Anhui University of Science and Technology, Huainan 232001, China; chenbinzhao96@aust.edu.cn
* Correspondence: shenwenting@qdu.edu.cn

Abstract

Clustered federated learning (CFL) addresses data heterogeneity by grouping clients with similar data distributions and training personalized models for each cluster. However, existing CFL schemes depend on clustering-related parameters that require prior knowledge of client data distributions. Such information is often unavailable in federated learning due to privacy constraints and the lack of efficient global coordination. This limitation becomes more severe under highly heterogeneous data distributions. In addition, privacy protection remains a major challenge since existing techniques often introduce substantial computational and communication overhead or significantly degrade model utility. To address these challenges, we propose PA-CFL, a novel privacy-preserving adaptive CFL framework under heterogeneous data. We introduce a novel structure-aware similarity metric to partition users based on their data distributions and decompose the clustering process into pre-clustering and fine clustering phases, thereby eliminating the need for manually specified clustering-related parameters. Furthermore, we propose a lightweight structure-aware privacy-preserving mechanism that decomposes each gradient into important and less important subvectors and applies differentiated protection to them. The less important subvector is protected through local randomization and intra-cluster shuffling with a formal differential privacy guarantee, whereas the important subvector is protected through permutation-based coordinate obfuscation under the non-collusion assumption. Extensive experimental results demonstrate that PA-CFL outperforms state-of-the-art methods in terms of model accuracy and efficiency while maintaining a favorable privacy–utility trade-off.

Keywords: federated learning; privacy-preserving; heterogeneous data

MSC: 68T05; 68P27

Check for updates

Academic Editor: Jonathan Blackledge

Received: 4 September 2026
Revised: 30 September 2026
Accepted: 5 October 2026
Published:

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Federated Learning (FL) enables multiple users to collaboratively train a global model without sharing their raw data [1]. In each training round, users perform local updates on their private datasets and upload the resulting model updates to a central server, which aggregates them to refine the global model [2,3]. However, FL's effectiveness can be limited by heterogeneous data among users, which often results in degraded global model performance and slower convergence [4,5]. Specifically, heterogeneous data is an inherent characteristic of FL, as users' local datasets are naturally diverse and differ in their statistical properties [6]. This inconsistency manifests as variations in label distributions or feature

spaces across users, leading to a statistically Non-Independent and Identically Distributed (Non-IID) data distribution among users [7,8].

To mitigate the adverse effects of heterogeneous data, numerous studies [9–11] have been proposed. A direct approach [9,12,13] is to reduce the discrepancies among users' local models to train a single global model that performs well across diverse data distributions. However, under highly heterogeneous settings, a single global model often fails to achieve satisfactory performance. To address this limitation, clustered federated learning (CFL) [10,14] has emerged as a promising solution. CFL partitions users into clusters with similar data distributions, allowing each cluster to train its own intra-cluster model. This approach effectively alleviates the impact of heterogeneity and improves both convergence and overall performance. Building on this idea, several CFL variants [11,15–18] have been proposed to achieve more efficient and accurate clustering. Despite these advances, the performance of existing CFL methods heavily depends on clustering-related parameters that typically require prior knowledge, such as the number of clusters or clustering thresholds. These parameters determine the clustering structure and are distinct from conventional hyperparameters used for model training or algorithm configuration. In practice, obtaining such prior knowledge is challenging in FL settings, as privacy constraints prevent direct access to users' local information, while the distributed nature of FL makes global coordination and iterative clustering-related parameter adjustment inefficient. This issue becomes more pronounced under highly heterogeneous data distributions. In such cases, inappropriate clustering-related parameter choices can lead to unstable clustering structures and incorrect user grouping, thereby exacerbating performance degradation and hindering model convergence. Consequently, the applicability of these methods is severely limited in realistic FL systems. This limitation highlights the first challenge: how to perform accurate and adaptive clustering without relying on manually specified clustering-related parameters.

Beyond heterogeneous data, privacy protection remains a critical challenge in FL. Prior studies [19–21] have demonstrated that local gradients shared during training can be exploited to infer sensitive information about local data. To address this issue, various privacy-preserving techniques have been incorporated into FL, including homomorphic encryption (HE) [22], secure multi-party computation (SMPC) [11], and differential privacy (DP) [23]. While these approaches provide privacy guarantees, they also introduce practical limitations. Specifically, DP may perturb model updates and degrade model accuracy, whereas cryptographic approaches such as HE and SMPC incur significant computational and communication overhead.

Although extensive efforts have been devoted to mitigating data heterogeneity or enhancing privacy protection individually, their joint consideration remains largely underexplored. Theoretically, integrating privacy-preserving techniques with CFL can simultaneously improve model performance under heterogeneous data while protecting user privacy. However, practical integration poses several challenges. For example, the noise introduced by DP may disrupt the similarity relationships among users, thereby degrading clustering performance, while cryptographic approaches (HE or SMPC) significantly increase system overhead, limiting their practicality in CFL. These observations highlight the second challenge: how to provide effective privacy protection while preserving clustering accuracy and maintaining system efficiency.

To tackle the above challenges, we propose PA-CFL, a novel privacy-preserving adaptive clustering federated learning framework for heterogeneous data. In summary, the contributions of our works are as follows:

- We propose a novel structure-aware similarity metric to measure user similarity based on the structural characteristics of their local gradients. Specifically, each user constructs a permutation sequence according to the importance of local gradient elements,

and user similarity is computed by applying Dynamic Time Warping (DTW) distance to the corresponding permutation sequences. This design avoids exposing raw gradient information while introducing low computational and communication overhead.

- We propose an adaptive clustering scheme that eliminates the need for manually specified clustering-related parameters. Specifically, a fast pre-clustering process is first performed based on the proposed similarity metric to automatically obtain preliminary clusters, followed by an iterative refinement process for more accurate partitioning. This approach avoids reliance on prior knowledge and repeated clustering-related parameter tuning, enabling robust clustering under highly heterogeneous data and improving model convergence.
- We propose a lightweight structure-aware privacy-preserving mechanism that leverages the importance-aware structure of local gradients to apply differentiated protection to gradient elements. Specifically, less important gradient elements are protected through local randomization and intra-cluster shuffling, while important gradient elements are safeguarded via permutation obfuscation. This mechanism effectively preserves privacy while mitigating the impact of perturbation on model accuracy.
- Convergence proofs and security analysis are provided to ensure the theoretical correctness of PA-CFL. In addition, extensive experiments are conducted on multiple datasets under various degrees of data heterogeneity and compared with existing methods, demonstrating superior performance. The computational and communication overheads of the proposed scheme are also analyzed, and the results show its efficiency.

2. Related Work

2.1. Privacy-Preserving Federated Learning

To safeguard data confidentiality, many studies have explored privacy-preserving FL schemes by integrating cryptographic techniques. Among these, HE has attracted particular attention due to its ability to perform homomorphic operations directly on ciphertexts. ShieldFL [24] utilized a Paillier-based dual-trapdoor scheme to enable homomorphic gradient computation. RFed [22] and FedPHE [25] both employed the CKKS homomorphic encryption scheme to achieve privacy protection. Although HE-based methods effectively preserve data privacy, they still encounter scalability challenges arising from ciphertext expansion and high computational complexity.

In parallel, SMPC has been widely applied in FL for its strong privacy-preserving capability, enabling collaborative model training without exposing users' raw data. In Trident [26], a four-party SMPC-based privacy-preserving framework is proposed. POSEIDON [27] introduces an SMPC-based privacy-preserving FL protocol for an arbitrary number of users. RTBF [28] employs a two-party computation protocol to balance security and efficiency. Although SMPC provides strong privacy protection, its multi-round communication requirements incur substantial overhead in practical deployments. Furthermore, DP been extensively explored as a lightweight approach to preserve data privacy in FL. Many federated learning approaches [23,29] employed DP to inject random noise into model updates. However, the injected noise inevitably reduces model accuracy. To mitigate the trade-off between privacy protection and model performance, FedSDP [30] employed the Shuffle DP(SDP) [31], which utilizes a semi-trusted shuffler that randomly permutes users' locally perturbed messages, thereby achieving privacy amplification through shuffling. To overcome the limitations of single-shuffler approaches, MSFL [32] introduced a multi-shuffler mechanism that leverages structured grouping and dynamic evaluation to enable more efficient and flexible privacy-preserving FL. Nevertheless, under severe heterogeneous data, local models may diverge, and the shuffling process can disrupt inter-user feature correlations, leading to higher aggregation errors and degraded global performance.

2.2. Clustered Federated Learning

To mitigate the adverse effects of heterogeneous data in FL, which often lead to degraded global model performance and hinder stable convergence, a lot of studies have explored optimization-based strategies to enhance training stability.

These effects include constraining local updates [33], promoting representation consistency [34], and calibrating outputs to alleviate class imbalance [35]. Although these methods improve global convergence, they inherently assume that a single global model can represent all user' diverse data distributions, which rarely holds in practice. The limited capacity of the global model and the divergence among heterogeneous user models often result in suboptimal aggregation performance and an inability to meet diverse user requirements.

To address this limitation, Sattler et al. [10] introduced the concept of clustered federated learning within the broader framework of federated multi-task learning, where users are partitioned into clusters with similar data distributions, allowing each cluster to train its own intra-cluster model. Building on this idea, IFCA [15] randomly initializes multiple cluster centers and assigns users to groups that minimize local training loss. PrivCrFL [11] replaced the costly iterative clustering process with a one-shot clustering strategy during the initial communication round, while FlexCFL [36] adopted K-means-based clustering to accelerate cold-start for new users, avoiding repeated iterations in hierarchical methods such as IFCA [15]. However, the performance of existing CFL methods heavily depends on clustering-related parameters that require prior knowledge, such as the number of clusters and clustering thresholds. In practical FL applications, acquiring such prior knowledge is challenging, since privacy constraints restrict access to users' local information, and the distributed nature of FL makes global coordination and iterative clustering-related parameter tuning inefficient, thereby limiting the applicability of these methods.

To achieve adaptive clustering under heterogeneous data environments while reducing the reliance on prior knowledge, AP-CFL [16] performed dynamic affinity propagation-based clustering in each communication round, removing the need to predefine the number of clusters. However, under severe heterogeneity, large discrepancies among user models lead to highly dispersed cosine similarities, preventing stable and accurate clustering.

Furthermore, the aforementioned CFL approaches overlook the issue of privacy protection. CFL inherently poses challenges for integration with existing privacy-preserving methods. Employing HE or SMPC incurs substantial computational or communication overhead, whereas incorporating DP noise during clustering may disrupt the similarity relationships among user models, thereby degrading model accuracy. Thus, developing a framework that simultaneously achieves accurate clustering and privacy protection with low computational and communication overhead remains a challenging problem, motivating the design of our scheme.

3. Preliminaries

3.1. Federated Learning

Federated Learning (FL) [1] is a distributed learning paradigm in which N users U_i ($i \in [1, N]$) collaboratively train a global model without sharing their local data. At the t-th iteration, the training process has three steps:

- Local training: In the t-th iteration, each user computes the gradient of the local model loss function on its private dataset D_i, thereby obtaining the local gradient $g_i^t = \nabla\mathcal{L}(\theta_i^{t-1}, D_i)$, where θ_i^{t-1} denotes the local model parameters of the U_i and $\nabla\mathcal{L}$ represents the standard SGD algorithm. Subsequently, the local gradient g_i^t is then transmitted to the server.

- Aggregation: After receiving local gradients $\{g_i^t\}_{i\in[1,N]}$, the server aggregates them to produce the global gradient $g^t = \frac{1}{N}\sum_{i=1}^{N} g_i^t$. The server delivers the global gradient g^t to all users.
- Model updates: After receiving the global model, each user updates its local model parameters θ_i^t using the global gradient g^t and the learning rate η: $\theta_i^t = \theta_i^{t-1} - \eta g^t$.

3.2. Differential Privacy and Laplace Mechanism

Differential Privacy (DP) provides a rigorous mathematical framework for quantifying the privacy guarantees of algorithms that operate on sensitive data. Two datasets $\mathcal{D}$ and $\mathcal{D}'$ in the domain $\mathcal{X}^n$ are defined as neighboring datasets if they differ by exactly one data record. The formal definition of DP is given as follows:

Definition 1. *(Differential Privacy-(ϵ,δ)-DP). A randomized mechanism $\mathcal{M}: \mathcal{X}^n \to \mathcal{Y}$ is said to satisfy (ϵ,δ)-DP if for any pair of neighboring datasets $\mathcal{D}, \mathcal{D}' \in \mathcal{X}^n$, and any measurable subset of outputs $\mathcal{V} \subseteq \mathcal{Y}$, the following inequality holds: $Pr[\mathcal{M}(\mathcal{D}) \in \mathcal{V}] \leq e^{\epsilon} Pr[\mathcal{M}(\mathcal{D}') \in \mathcal{V}] + \delta$.*

Here the parameter ϵ denotes the privacy budget, where a smaller value results in stronger privacy guarantees and the parameter δ is a small probability of privacy violation. In PA-CFL, privacy-preserving randomization is performed locally before the corresponding gradient information is transmitted. Therefore, we further introduce local differential privacy (LDP), which characterizes the privacy guarantee of a local randomizer.

Definition 2. *(Local Differential Privacy-$(\boldsymbol{\epsilon,\delta})$-LDP). A randomized mechanism $\mathcal{R}: \mathcal{X} \to \mathcal{Y}$ satisfies (ϵ,δ)-LDP if, for any two possible inputs $x, x' \in \mathcal{X}$ and any measurable subset $\mathcal{V} \subseteq \mathcal{Y}$,*

$$Pr[\mathcal{R}(x) \in \mathcal{V}] \leq e^{\epsilon} Pr[\mathcal{R}(x') \in \mathcal{V}] + \delta.$$

PA-CFL employs the Laplace mechanism as the local randomizer for the less important gradient components. For a scalar function $f(\cdot)$, its sensitivity is defined as $\Delta_f = \max_{x,x'\in\mathcal{X}} |f(x) - f(x')|$. Given a local privacy budget ϵ_0, the Laplace mechanism is defined as

$$\mathcal{R}(x) = f(x) + \xi, \qquad \xi \sim \mathrm{Lap}\left(0, \frac{\Delta_f}{\epsilon_0}\right).$$

With this noise scale, the resulting coordinate-wise randomizer satisfies $(\epsilon_0, 0)$-LDP. In PA-CFL, the Laplace mechanism is independently applied to each coordinate of the clipped less important gradient subvector. After local randomization, the resulting reports are further shuffled within each pre-cluster to amplify privacy.

3.3. Dynamic Time Warping and FastDTW

Dynamic Time Warping (DTW) is a classical method for measuring the similarity between two sequences and is particularly effective for aligning sequences with nonlinear structural variations.

Given two sequences $X = \{x_1, x_2, \ldots, x_m\}$ and $Y = \{y_1, y_2, \ldots, y_n\}$, DTW seeks an optimal warping path $P = \{(i_1, j_1), (i_2, j_2), \ldots, (i_{|P|}, j_{|P|})\}$ that aligns the two sequences from $(1,1)$ to (m,n). The DTW distance is defined as the minimum cumulative alignment cost along all feasible warping paths:

$$DTW(X,Y) = \min_{P} \sum_{k=1}^{|P|} d(x_{i_k}, y_{j_k}),$$

where $d(\cdot,\cdot)$ denotes the local distance function. Compared with point-wise distance measures, DTW can better capture structural similarity by allowing flexible local alignment.

Standard DTW searches over the complete $m \times n$ cost matrix and therefore requires $O(mn)$ time and space. To reduce this overhead, FastDTW [37] adopts a multilevel coarse-to-fine alignment strategy. Let $X^{(l)}$ and $Y^{(l)}$ denote the two sequences at resolution level l, where a larger l represents a coarser resolution. At each level, the sequences are reduced by $X^{(l+1)} = \text{Reduce}(X^{(l)})$ and $Y^{(l+1)} = \text{Reduce}(Y^{(l)})$, where $\text{Reduce}(\cdot)$ denotes the down-sampling operation that constructs a lower-resolution sequence. FastDTW recursively computes an approximate warping path $P^{(l+1)}$ between the two reduced sequences at level $l+1$. This path is then mapped back to level l through the projection operation $\text{Proj}(\cdot)$. Based on the projected path, FastDTW constructs a restricted search region $\mathcal{W}^{(l)}$ as $\mathcal{W}^{(l)} = \{(i,j) \mid \text{dist}((i,j), \text{Proj}(P^{(l+1)})) \leq r\}$, where (i,j) denotes a pair of positions in $X^{(l)}$ and $Y^{(l)}$, $\text{dist}(\cdot,\cdot)$ denotes the distance from (i,j) to the projected warping path, and r is a fixed search radius controlling the width of the search region. Within $\mathcal{W}^{(l)}$, the accumulated alignment cost is computed as

$$D(i,j) = d(x_i, y_j) + \min\{D(i-1,j), D(i,j-1), D(i-1,j-1)\},$$

where $D(i,j)$ denotes the minimum accumulated cost for aligning the first i and j elements of the two sequences, and $d(x_i, y_j)$ denotes the local distance between elements x_i and y_j. By restricting DTW computation to the neighborhood of the projected path, FastDTW avoids evaluating the complete cost matrix. For two sequences of length k and a fixed search radius r, its time and space complexities scale as $O(k)$.

3.4. Davies–Bouldin Index

The Davies–Bouldin Index (DBI) is an internal evaluation metric used to assess the quality of clustering results [38]. Suppose the N users U_i ($i \in [1, N]$) are partitioned into K clusters, denoted by $\mathcal{C} = \{C_1, C_2, \ldots, C_K\}$. Let $Dist \in \mathbb{R}^{N \times N}$ denote the pairwise distance matrix among all users, where $[Dist]_{pq}$ represents the distance between users U_p and U_q. For each cluster C_i, we define its representative center as the medoid, i.e., the user whose average distance to all other users in the same cluster is minimal. Formally, the medoid of cluster C_i is defined as

$$\mu_i^* = \arg\min_{U_l \in C_i} \frac{1}{|C_i|} \sum_{U_j \in C_i} [Dist]_{lj}.$$

Based on the medoid, the intra-cluster dispersion of cluster C_i is defined as

$$S_i = \frac{1}{|C_i|} \sum_{U_j \in C_i} [Dist]_{j\,\mu_i^*}$$

which measures the average distance between users in C_i and their cluster medoid. For two clusters C_i and C_j, the inter-cluster separation is defined as the distance between their medoids: $M_{ij} = [Dist]_{\mu_i^*\,\mu_j^*}$. Then, DBI is computed as

$$DBI = \frac{1}{K} \sum_{i=1}^{K} \max_{i \neq j} \left(\frac{S_i + S_j}{M_{ij}} \right).$$

A smaller DBI indicates better clustering quality, corresponding to more compact intra-cluster structure and better separation between different clusters.

4. Scenario Description

4.1. System Model

As shown in Figure 1, there are three basic entities in PA-CFL:

- Users (U): Each user trains a local model on its private dataset and obtains a local gradient. It then generates a permutation sequence and uploads it to the MS. After receiving the global permutation sequence, each user decomposes its local gradient into two subvectors. The important gradient subvector is protected by permutation obfuscation, while the less important gradient subvector is local randomized. The resulting important gradient subvector and randomized less important gradient subvector are uploaded to the AS and MS, respectively.
- Management server (MS): The MS aggregates users' permutation sequences to derive a global permutation sequence and broadcasts it to all users. It then performs coarse pre-clustering to obtain a preliminary clustering structure and forwards it to the AS. In addition, the MS executes intra-shuffling to the randomized less important gradient subvectors and sends the results to the AS.
- Aggregation server (AS): The AS concatenates the important gradient subvectors from users with the shuffled less important gradient subvectors from the MS, thereby reconstructing the privacy-preserving gradient for each user. Based on the coarse pre-clusters, the AS performs fine-clustering to assign users to different clusters. It then aggregates the privacy-preserving gradients within each cluster according to the refined clustering structure. Finally, the AS returns either the intra-cluster aggregated gradient or the global aggregated gradient to each user.

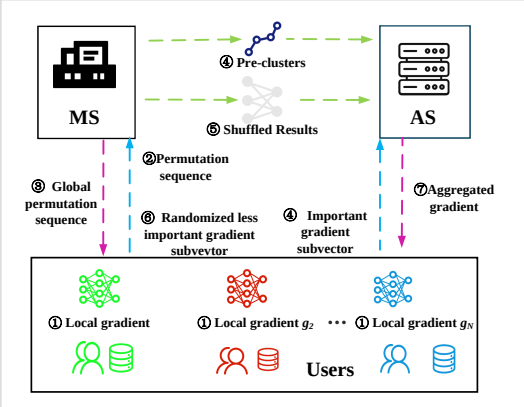


Figure 1. System model of PA-CFL.

4.2. Threat Model

Following existing works [39–42], PA-CFL adopts the following threat model:

- Honest-but-Curious Users: All users faithfully follow the protocol but may attempt to infer private information about others. PA-CFL allows $N-1$ users to collude with either the AS or the MS, where N is the total number of users.
- Honest-but-Curious and Non-Colluding Servers: Consistent with prior dual-server settings [22,43,44], we also assume that both the AS and the MS are honest-but-curious and non-colluding. That is, they honestly execute the protocol while being interested in users' private information.

4.3. Designed Goals

The designed goals of PA-CFL are as follows:

- Privacy protection: PA-CFL protects each user's local gradient through a combination of permutation obfuscation, local randomization, and intra-cluster shuffling. These mechanisms prevent the AS, the MS, or any colluding users from inferring users' private information.
- Clustering effectiveness and system efficiency: PA-CFL employs an adaptive clustering algorithm that automatically and accurately assigns users based on their data distributions, without requiring prior knowledge of clustering-related parameters. Moreover, the lightweight privacy-preserving mechanism introduce low computational and communication overhead, improving overall system efficiency.
- Robustness to heterogeneous data: PA-CFL maintains high model accuracy and robust performance under a variety of non-IID data distributions, making it suitable for practical applications.

5. Design of PA-CFL

5.1. Overview

CFL aims to mitigate data heterogeneity by partitioning users into groups with similar data distributions. However, existing CFL solutions suffer from two major limitations. First, their performance often depends on manually specified clustering-related parameters or prior assumptions about user data distributions, which is impractical in real-world federated learning systems where such information is unavailable and privacy constraints limit access to local data. Second, most existing methods do not adequately protect user privacy during similarity computation and clustering, which may lead to potential information leakage.

To address these limitations, we propose PA-CFL, a privacy-preserving adaptive clustered federated learning framework for heterogeneous data. The key idea is to exploit gradient structural information for user clustering, while applying differentiated protection mechanisms to gradient components based on their importance in clustering and optimization.

The motivation for exploiting gradient structural information is illustrated in Figure 2. Although the numerical gradient values vary across users, high-magnitude gradient elements tend to concentrate on a limited set of coordinates, forming distinguishable structural patterns. These patterns reflect which model parameters are more strongly affected by each user's local data. Therefore, instead of directly comparing complete numerical gradients, PA-CFL extracts the coordinate indices and relative ranking of high-importance gradient elements as a compact structural representation for measuring user similarity.

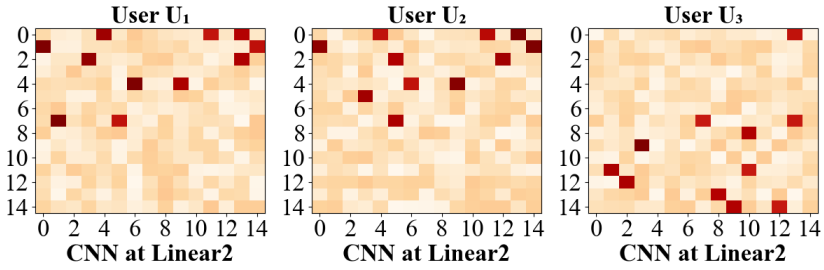


Figure 2. Heatmaps of local gradients at the second linear layer of a CNN trained on non-IID MNIST across three users. Darker colors indicate larger gradient element values.

Meanwhile, gradient components with different importance need not receive the same privacy treatment. High-importance gradient elements contain more useful optimization information and are further used in the subsequent clustering refinement. Applying strong perturbations to these elements may reduce clustering accuracy and degrade model

utility. In contrast, less important components have a relatively smaller influence on optimization and can tolerate stronger perturbation. Accordingly, PA-CFL protects important components through permutation obfuscation, while applying local randomization and intra-cluster shuffling to less important components.

Figure 3 illustrates the overall workflow of PA-CFL. The framework consists of six sequential phases. The first three phases extract gradient structural information and obtain a preliminary cluster structure, while the latter three phases perform differentiated privacy protection, clustering refinement, and model aggregation.

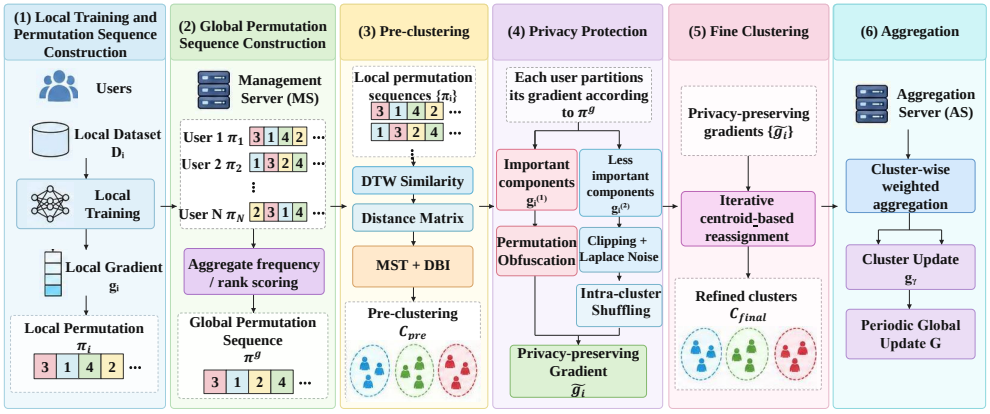


Figure 3. Conceptual workflow of PA-CFL.

- Local Training and Permutation Sequence Construction. Each user computes its local gradient g_i and extracts the top-k gradient coordinates ranked by absolute magnitude to construct a permutation sequence π_i, which serves as a compact structural representation of the local gradient.
- Global Permutation Sequence Construction. The MS aggregates the local permutation sequences $\{\pi_i\}$ to construct a global permutation sequence π^g, which identifies gradient coordinates that are consistently important across users and guides subsequent gradient partitioning.
- Pre-clustering. The MS computes pairwise DTW distances between local permutation sequences and performs MST-based partitioning with a DBI-guided criterion to obtain the preliminary clustering result C_{pre}.
- Privacy Protection. Guided by π^g, each local gradient is divided into important and less important subvectors. The former is protected by permutation obfuscation, while the latter is protected by local randomization and intra-cluster shuffling, producing the privacy-preserving gradient $\widetilde{g}_i$.
- Fine Clustering. Using $\{\widetilde{g}_i\}$ and C_{pre}, the AS iteratively refines user assignments to obtain the final clustering result C_{final}.
- Aggregation. Based on C_{final}, the AS performs cluster-wise weighted aggregation and periodically performs global aggregation to maintain information exchange across clusters.

In summary, PA-CFL first exploits compact gradient structural information to identify users with similar update characteristics and then uses the resulting structural information to guide differentiated privacy protection. The preliminary clustering result provides the grouping basis for intra-cluster shuffling, while the privacy-preserving gradients support subsequent fine clustering and aggregation. Therefore, the six phases form a tightly coupled workflow that jointly balances clustering effectiveness, privacy protection, and model utility.

5.2. Local Training and Permutation Sequence Construction

Assume there are N participating users, denoted by $U = \{U_1, U_2, \ldots, U_N\}$. Each user U_i performs local training to generate a privacy-preserving structural representation of its gradient. The process contains two steps: computing the local gradient and constructing the permutation sequence.

Local Training:

Each U_i performs local training on its private dataset $\mathcal{D}_i$ and computes the corresponding local gradient $g_i = \nabla\mathcal{L}(\theta_i^{t-1}, \mathcal{D}_i)$, where θ_i^{t-1} denotes the local model parameters from the previous round $t-1$ and $\nabla\mathcal{L}(\cdot)$ represents the gradient of the loss function with respect to these parameters.

Permutation Sequence Construction:

To extract a compact structural representation, user selects the top-k most important gradient elements in magnitude and constructs a permutation sequence from their coordinate indices. The permutation sequence π_i is computed as

$$\pi_i = \text{argsort}\big(top-k(flatten(g_i))\big),$$

where $flatten(\cdot)$ converts the gradient tensor into a one-dimensional vector, $top\text{-}k(\cdot)$ returns the indices of the k elements with the largest absolute gradient magnitudes (with $k \ll d$, where d is the gradient dimension), together with their corresponding values, and $argsort(\cdot)$ sorts these selected elements in descending order of absolute values and returns their corresponding coordinate indices in the original gradient g_i. Each user U_i uploads its permutation sequence π_i to the MS.

Figure 4 provides an example demonstrating how π_i is constructed with gradient dimension $d = 6 \times 6$. Operation ② ($top\text{-}k(\cdot)$) selects the top $k = 0.2 \times d = 7$ elements, forming the coordinate index set $(1, 3, 5, 10, 19, 21, 22)$ and the corresponding value set $X = (9, 8, 7, -5, -8, 4, 3)$, where each element in X is aligned with its index in the same position. The dashed box in Figure 4 highlights their coordinate indices in the flattened vector. For example, the element "4" originally at coordinate (4, 3) in the tensor corresponds to coordinate index 21 after flattening. Operation ③ ($argsort$) sorts the elements in X in descending order of their absolute values and reorders the corresponding coordinate indices accordingly, yielding $\pi_i = (1, 3, 19, 5, 10, 21, 22)$.

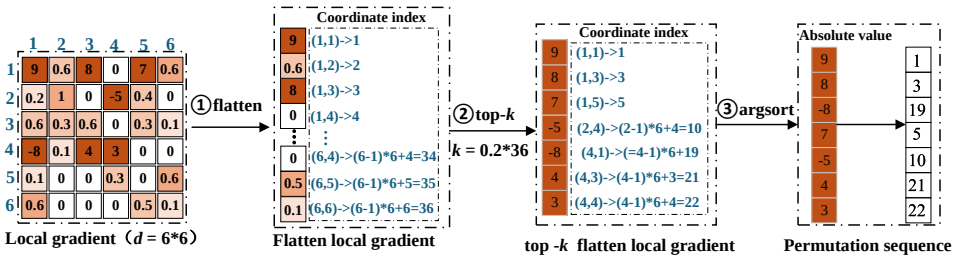


Figure 4. Permutation sequence computation schematic.

5.3. Global Permutation Sequence Construction

After receiving permutation sequences $\{\pi_i\}_{i\in[1,N]}$ from users, the MS constructs a global permutation sequence that captures the shared structural patterns of local gradients across users. The construction procedure has three steps:

Importance scoring:

For each coordinate index $e \in [1, d]$ that may appear in users' permutation sequences, MS evaluates its importance score based on both its frequency and ranking position across all users. Since elements appearing earlier in the permutation sequence π_i correspond to gradients with larger magnitude, a coordinate index receives a higher contribution when it appears closer to the front of a permutation sequence π_i. The importance score of coordinate index e is defined as

$$p(e) = \sum_{i=1}^{N} \frac{1_{e \in \pi_i}}{1 + j_i(e)}$$

where $j_i(e)$ denotes the position of e in the permutation sequence π_i, which is defined only when $e \in \pi_i$, and $1_{e \in \pi_i} = 1$ if e exists in π_i, otherwise 0. This scoring mechanism ensures that coordinate indices frequently selected by users and ranked in earlier positions receive higher importance.

Construct Preliminary Global Permutation Sequence:

The MS then sorts all collected coordinate indices in descending order of their importance scores $p(e)$: $\pi' = \text{sort}(e \text{ by } p(e))$, where $\text{sort}(\cdot)$ orders the coordinate indices according to their associated importance scores. This yields a preliminary global permutation sequence π'.

Sequence Truncation:

To obtain a compact global structural representation, the MS retains the first k elements of the preliminary global permutation sequence π', resulting in the final global permutation sequence π^g. Here, k is consistent with the top-k used by each user when constructing its local permutation sequence π_i. This truncation reduces construction complexity while preserving the coordinate indices of the high-importance gradient elements.

5.4. Pre-Clustering

The pre-clustering stage aims to obtain an initial grouping of users from their gradient structural relationships without prior knowledge of the number of clusters. PA-CFL first measures the structural similarity between users based on their local permutation sequences and then progressively partitions the resulting user relationship structure according to the observed separation among users. The quality of each partition is further evaluated to determine whether additional splitting is necessary.

After obtaining the permutation sequence π_i for each user, the MS performs pre-clustering based on gradient structural similarity. This phase consists of two steps as follows:

Structure-aware Similarity Metric:

The permutation sequence π_i characterizes the dominant gradient structure of user U_i by recording the coordinate indices of its top-k gradient elements in descending order of magnitude. Users with similar local data distributions are expected to exhibit similar dominant gradient structures, as their local training tends to produce similar patterns of parameter importance. Therefore, PA-CFL measures the similarity between users by comparing their permutation sequences. DTW is employed for this comparison because it measures the overall structural discrepancy between two ordered sequences while accommodating variations in the ranking positions of individual gradient coordinates.

To quantify the gradient structural similarity between users U_i and U_j, let $\pi_i = (e_{i,1}, e_{i,2}, \ldots, e_{i,k})$ and $\pi_j = (e_{j,1}, e_{j,2}, \ldots, e_{j,k})$ denote their permutation sequences, where $e_{i,r}$ is the coordinate index of the r-th largest gradient element in $|g_i|$. When applying DTW

to π_i and π_j, the local distance between $e_{i,r}$ and $e_{j,s}$ is defined as $d(e_{i,r}, e_{j,s}) = |e_{i,r} - e_{j,s}|$. Accordingly, the structural distance between U_i and U_j is computed as

$$\rho_{ij} = DTW(\pi_i, \pi_j) = \min_{P} \sum_{(r,s)\in P} d(e_{i,r}, e_{j,s}),$$

where P denotes a feasible warping path. A smaller ρ_{ij} indicates higher similarity between their dominant gradient structures.

Pre-clustering Process:

Using the DTW-based structural distance, the MS computes the pairwise distances between users and obtains a symmetric distance matrix $Dist \in \mathbb{R}^{N\times N}$, where the (i, j)-th entry is $[Dist]_{ij} = \rho_{ij} = DTW(\pi_i, \pi_j), \quad (i, j \in \{1, 2, \ldots, N\})$.

Based on the precomputed distance matrix $Dist$, the MS constructs a minimum spanning tree (MST) over all users U, where ρ_{ij} is used as the edge weight between users U_i and U_j. Formally, the MST is denoted as $\mathcal{T} = (U, \mathcal{E})$, where $\mathcal{E}$ is the edge set of $\mathcal{T}$. The MST is a connected acyclic graph that spans all users in U with exactly $N - 1$ edges. Among all spanning trees over U, it minimizes the total edge weight:

$$\mathcal{T} = \operatorname*{argmin}_{\mathcal{T}'=(U,\mathcal{E}')\in\mathcal{S}} \sum_{e_{ij}\in\mathcal{E}'} \rho_{ij},$$

where $\mathcal{S}$ denotes the set of all spanning trees over U, $\mathcal{E}'$ is the edge set of spanning trees $\mathcal{T}'$, and e_{ij} represents the edge between users U_i and U_j.

Before the iterative edge removal process, the MS initializes the best DBI value as $DBI^{best} = +\infty$. The edges in $\mathcal{E}$ are implicitly processed in descending order of their weights through iterative maximum-edge removal.

To derive cluster structures, the MS progressively removes edges with large weights from the MST, as such edges typically connect weakly related users. As edges are removed, the MST decomposes into multiple connected components, each corresponding to a cluster. This process proceeds iteratively as follows:

- At each iteration, the MS identifies the edge with the largest weight: $e^\star = \arg\max_{e_{ij}\in\mathcal{E}} \rho_{ij}$, removes it to obtain a new edge set $\mathcal{E}' = \mathcal{E} \setminus \{e^\star\}$, which induces a set of connected components $\mathcal{Y}'$, where each element $Y \in \mathcal{Y}'$ corresponds to a connected component, i.e., a connected subset of users.
- After each edge removal, the MS evaluates whether the operation is valid based on the following two criteria: (a) Each connected component $Y \in \mathcal{Y}'$ must satisfy $|Y| \geq 2$. (b) If the size constraint is satisfied, the MS evaluates the clustering quality of $\mathcal{Y}'$ using the Davies–Bouldin Index (DBI). Specifically, based on the distance matrix $Dist$, each cluster $C_i \in \mathcal{Y}'$ is represented by its medoid:

$$\mu_i^* = \arg\min_{U_l\in C_i} \frac{1}{|C_i|} \sum_{U_j\in C_i} [Dist]_{lj}.$$

The intra-cluster dispersion is computed as the average distance between users and their cluster medoid, i.e.,

$$S_i = \frac{1}{|C_i|} \sum_{U_j\in C_i} [Dist]_{j\,\mu_i^*},$$

and the inter-cluster separation between clusters C_i and C_j is defined as $M_{ij} = [Dist]_{\mu_i^*\,\mu_j^*}$. Based on these quantities, the DBI value of $\mathcal{Y}'$ is computed according to the definition in Section III-D. The removal is accepted only if the resulting DBI value does not exceed the current best value DBI^{best}.

If the removal passes the validity check, the MS updates $\mathcal{E} \leftarrow \mathcal{E}'$, $\mathcal{Y} \leftarrow \mathcal{Y}'$, and $DBI^{best} \leftarrow DBI(\mathcal{Y}')$. DBI jointly considers intra-cluster compactness and inter-cluster separation based on the DTW distance matrix. Therefore, a decrease in DBI indicates that the candidate partition forms more compact clusters with clearer separation, whereas an increase suggests that further splitting does not improve the current clustering structure. By accepting an edge removal only when it does not increase the best DBI value, PA-CFL adaptively determines whether further partitioning is necessary without requiring a predefined number of clusters. The process continues until no further edge removal satisfies the validity criteria. Finally, the resulting connected component set $\mathcal{Y}$ is taken as the pre-clustering result, denoted by C_{pre}.

5.5. Privacy Protection

Each user first partitions its local gradient into important and less important gradient elements under the guidance of the global permutation sequence π^g. Specifically, the gradient elements indexed by π^g are regarded as globally important components and are used to form the important gradient subvector, while the remaining elements constitute the less important gradient subvector. Let d_1 and d_2 denote the dimensionalities of the important and less important gradient subvectors, respectively, with $d_1 + d_2 = d$. We next detail the distinct protection strategies for these two gradient subvectors.

Protection of Important Gradient Subvectors:

After receiving the global permutation sequence π^g, each user constructs the important gradient subvector by selecting and ordering the elements of its local gradient according to π^g, forming the globally important gradient components:

$$\widetilde{g}_i^{(1)} = \{g_i[\pi^g[1]], g_i[\pi^g[2]], \ldots, g_i[\pi^g[|\pi^g|]]\} = \pi^g \circ g_i.$$

where $\pi^g[l]$ denotes the l-th element in π^g, and $\circ$ denotes the permutation operation that reorders g_i according to π^g, thereby obfuscating the original positions of important gradient elements for privacy protection. Then, user U_i uploads the important gradient subvector $\widetilde{g}_i^{(1)}$ to the AS for subsequent fine clustering.

Protection of Less Important Gradient Subvectors:

The remaining gradient elements, whose indices are not included in π^g, preserve their original order in the local gradient g_i and form the less important gradient subvector: $g_i^{(2)} = (g_i[j])_{j \notin \pi^g}$. To protect $g_i^{(2)}$, we adopt a two-stage privacy-preserving mechanism consisting of local randomization and cluster-based shuffling.

(1) Local randomization: This stage consists of two steps:

- To bound the influence of each user update and the coordinate-level sensitivity of the subsequent Laplace mechanism, each user U_i first clips its less important gradient subvector $g_i^{(2)}$ as

$$\bar{g}_i^{(2)} = g_i^{(2)} \cdot \min\left(1, \frac{\Phi}{\|g_i^{(2)}\|}\right),$$

where $\Phi > 0$ is the clipping threshold and $\|\cdot\|$ denotes the L_2-norm.
This operation ensures that $\|\bar{g}_i^{(2)}\| \leq \Phi$ and hence $|\bar{g}_i^{(2)}[j]| \leq \Phi$ for every coordinate j. Therefore, each coordinate lies in the interval $[-\Phi, \Phi]$, and its maximum possible change between two clipped inputs is bounded by

$$\Delta_f = \max\left|\bar{g}_i^{(2)}[j] - \bar{g}_i^{(2)\prime}[j]\right| \leq 2\Phi,$$

where $\bar{g}_i^{(2)}$ and $\bar{g}_i^{(2)\prime}$ denote any two possible clipped less important gradient subvectors.

- After clipping, each user U_i applies a Laplace mechanism $\mathcal{R}(\cdot)$ to the clipped less important gradient subvector $\bar{g}_i^{(2)}$, obtaining the randomized less important gradient subvector:

$$g_i^{(2)'} = \mathcal{R}\left(\bar{g}_i^{(2)}\right) = \bar{g}_i^{(2)} + \xi_i.$$

where $\xi_i \in \mathbb{R}^{d_2}$ is a noise vector whose coordinates are independently sampled as $\xi_i[j] \sim \mathrm{Lap}\left(0, \frac{\Delta_f}{\epsilon_0}\right)$, Δ_f denotes the coordinate-level sensitivity bound and ϵ_0 denotes the local privacy budget assigned to each coordinate. Accordingly, each coordinate-wise local randomizer satisfies $(\epsilon_0, 0)$-LDP.
Then, each user U_i sends the randomized less important gradient subvector $g_i^{(2)'}$ to the MS.

(2) Cluster-based shuffling: After receiving the randomized less important gradient subvectors $\{g_i^{(2)'}\}_{U_i \in C}$ for each pre-cluster $C \in C_{pre}$, the MS performs intra-cluster shuffling to further enhance privacy. Users within each cluster C are arranged in a fixed deterministic order. Let $idx(z)$ denote the global index of the z-th user in C, i.e., the z-th user is $U_{idx(z)}$. Let $pos(i)$ denote the position of user U_i in this ordering.

- For each coordinate $j \in \{1, 2, \ldots, d_2\}$, the MS collects the j-th elements of all randomized less important gradient subvectors in cluster C and constructs a vector:

$$V^{(j)} = (V^{(j)}[1], V^{(j)}[2], ..., V^{(j)}[|C|]) \in \mathbb{R}^{|C|},$$

where $V^{(j)}[z] = g_{idx(z)}^{(2)'}[j] (z \in \{1, 2, \ldots, |C|\})$.

- The MS samples an independent random permutation $\pi^{(j)}$ over $\{1, 2, \ldots, |C|\}$ for each coordinate $j \in \{1, 2, \ldots, d_2\}$, and applies it to $V^{(j)}$, yielding the shuffled vector:

$$\widetilde{V}^{(j)} = (\widetilde{V}^{(j)}[1], \widetilde{V}^{(j)}[2], ...\widetilde{V}^{(j)}[|C|])$$

where $\widetilde{V}^{(j)}[z] = V^{(j)}\left[\pi^{(j)}(z)\right] (z \in \{1, 2, \ldots, |C|\})$.

- For each user $U_i \in C$, the MS constructs its shuffled less important gradient subvector by taking, at each coordinate j, the entry of $\widetilde{V}^{(j)}$ corresponding to position $pos(i)$, i.e.,

$$\widetilde{g}_i^{(2)} = \left(\widetilde{V}^{(1)}[pos(i)], \widetilde{V}^{(2)}[pos(i)], \ldots, \widetilde{V}^{(d_2)}[pos(i)]\right).$$

After completing the shuffling procedure within each cluster, the MS sends $\{\widetilde{g}_i^{(2)}\}_{i=1}^N$ to the AS for subsequent aggregation. This operation decorrelates cross-user alignment of less important gradient components while preserving intra-cluster statistical structure.

5.6. Fine Clustering

Based on the coarse structure provided by the pre-clustering phase, the AS performs a refined clustering procedure to further improve clustering accuracy. The process of fine clustering consists of the following four steps:

Initialization:

Based on the received the important gradient subvectors $\{\widetilde{g}_i^{(1)}\}_{i \in [1,N]}$ and the shuffled results $\{\widetilde{g}_i^{(2)}\}_{i \in [1,N]}$, the AS reconstructs the complete privacy-preserving gradient for each

user U_i: $\widetilde{g}_i = \widetilde{g}_i^{(1)} || \widetilde{g}_i^{(2)}$, where $||$ denotes vector concatenation. The AS initializes the clustering using the pre-clustering result, i.e., $C_{old} \leftarrow C_{pre}$ and $C_{new} \leftarrow \phi$. This ensures that the refined clustering begins from a reliable, structure-preserving baseline produced by the coarse pre-clustering phase. At each iteration, C_{old} represents the cluster assignment from the previous iteration, while C_{new} denotes the updated assignment generated in the current iteration.

After each refinement step, the update $C_{old} \leftarrow C_{new}$ is performed before entering the next iteration, so that the clustering structure obtained in the current round serves as the reference for the next update. The refinement process iteratively updates the clustering structure until convergence.

Cluster Centroid Computation:

Given the current cluster assignment C_{old}, the AS computes the centroid for each cluster in C_{old}. Since users with larger datasets generally yield more reliable gradients, we assign each user $U_i \in C_y$ a weight proportional to its local dataset size:

$$w_i' = \frac{|\mathcal{D}_i|}{\sum_{U_j \in C_y} |\mathcal{D}_j|},$$

where $\mathcal{D}_i$ denotes the local dataset of user U_i.

The centroid of cluster C_y ($y \in \{1, 2, ..., |C_{old}|\}$) is then defined as the weighted aggregation of user gradients: $p_y = \sum_{U_i \in C_y} w_i' \widetilde{g}_i$, where $|C_{old}|$ denotes the number of clusters in the current assignment. This weighting scheme ensures that users with larger datasets contribute more to the cluster representation, resulting in a more stable centroid estimation.

User Reassignment:

Let the set of centroids be $P = \{p_y \mid y \in \{1, 2, \ldots, |C_{old}|\}\}$, where each centroid p_y corresponds one-to-one to cluster C_y. Each user is assigned to the cluster whose centroid is closest to its gradient representation. Formally, $x_i = \arg\min_{1 \le y \le |C_{old}|} \left(1 - \cos(\widetilde{g}_i, p_y)\right)$, where x_i denotes the index of the cluster assigned to user U_i, i.e., $U_i \in C_{x_i}$. That is, user U_i is assigned to cluster C_{x_i}, where x_i identifies the centroid that minimizes the cosine distance to $\widetilde{g}_i$. Here, $1 - \cos(\cdot)$ is used as a similarity-induced distance metric, where cosine similarity measures directional alignment between gradient representations.

After all users are reassigned, the updated clustering result is obtained as $C_{new} = \{C_1^{new}, C_2^{new}, \ldots, C_{|C_{old}|}^{new}\}$. Empty clusters, if any, are removed from the clustering set.

Iterative Refinement and Convergence:

Steps Cluster centroid computation and User Reassignment are repeated iteratively until convergence, i.e., when the cluster assignment satisfies $C_{new} = C_{old}$. This indicates that no user changes its cluster assignment between two consecutive iterations, and thus a fixed-point clustering structure has been reached. The final clustering result is denoted as C_{final}.

5.7. Aggregation

The AS performs weight aggregation within each cluster as the follow three steps:

Softmax-based Weight Assignment:

Once the refined clustering configuration C_{final} is obtained, the AS performs weight assignment within each cluster. For each cluster $C_y \in C_{final}$ and each user $U_i \in C_y$, the aggregation weight is computed by applying a softmax normalization over the cosine similarities between the privacy-preserved gradients $\widetilde{g}_i$ and the corresponding cluster centroid p_y:

$$w_i = \frac{\exp(\cos(\widetilde{g}_i, p_y))}{\sum_{U_j \in C_y} \exp(\cos(\widetilde{g}_j, p_y))}.$$

Here, $\exp(\cdot)$ denotes the exponential function.

Intra-cluster Aggregation:

The aggregated gradient for the cluster C_y is computed as

$$g_y = \sum_{U_i \in C_y} w_i \widetilde{g}_i.$$

Global Aggregation:

To preserve cross-cluster generalization and avoid overfitting to individual clusters, the AS periodically computes a global aggregated gradient:

$$G = \sum_{C_y \in C_{final}} \frac{|C_y|}{\sum_{C_j \in C_{final}} |C_j|} \cdot \frac{g_y}{\|g_y\|}.$$

The normalization $\frac{g_y}{\|g_y\|}$ is introduced to align gradient directions across clusters while preventing clusters with large magnitudes from dominating the global update. Depending on the training round t, the AS determines the aggregation strategy as follows: If $t \bmod \kappa \neq 0$, the intra-cluster aggregated gradient g_y is sent to users in cluster C_y. If $t \bmod \kappa = 0$, the global aggregated gradient G is broadcast to all users. Here, κ denotes a predefined synchronization period controlling the frequency of global aggregation.

6. Theoretical Analysis

6.1. Privacy Analysis: Differential Privacy and Coordinate Obfuscation

PA-CFL provides two distinct levels of privacy protection for different gradient components. The formal differential privacy guarantee applies only to the less important gradient subvector, which is protected through local randomization and intra-cluster shuffling. In contrast, the important gradient subvector is protected through permutation-based coordinate obfuscation under the non-collusion assumption. We first characterize the information exposed to the MS and AS during protocol execution and then analyze the corresponding privacy guarantees. For a user U_i, we summarize the information relevant to the two servers as

$$\text{View}_{\text{MS}}(U_i) = \left\{\pi_i, g_i^{(2)'}\right\}, \quad \text{View}_{\text{AS}}(U_i) = \left\{\tilde{g}_i^{(1)}, \tilde{g}_i^{(2)}\right\}.$$

These views are used below to distinguish the formal differential privacy guarantee from the information-exposure properties of the protocol. We provide an explicit differential privacy guarantee for the less important gradient subvector. For each user U_i, the local gradient is decomposed as $g_i = (g_i^{(1)}, g_i^{(2)})$, where $g_i^{(2)} \in \mathbb{R}^{d_2}$ denotes the less important gradient subvector. We adopt user-level adjacency over the less important gradient reports: two input collections are adjacent if they differ in the entire less important gradient subvector $g_i^{(2)}$ of a single user U_i.

After clipping, each coordinate of the clipped less important gradient subvector $\bar{g}_i^{(2)}$ is first locally randomized and then anonymously shuffled within the same coordinate group across users. For each coordinate j, the local randomizer is defined as

$$\mathcal{R}\left(\bar{g}_i^{(2)}[j]\right) = \bar{g}_i^{(2)}[j] + \xi_i[j], \qquad \xi_i[j] \sim \text{Lap}\left(0, \frac{\Delta_f}{\epsilon_0}\right),$$

where Δ_f denotes the coordinate-level sensitivity bound induced by clipping and ϵ_0 denotes the local privacy budget assigned to each coordinate. The noise terms added to different coordinates are independently sampled. Accordingly, the coordinate-wise local randomization mechanism satisfies $(\epsilon_0, 0)$-LDP. The resulting randomized less important gradient subvector is denoted by $g_i^{(2)'} = \mathcal{R}(\bar{g}_i^{(2)})$.

Theorem 1. *Fix a coordinate j and consider the n users in a cluster. Assume that*

(1) The clipped coordinate $\bar{g}_i^{(2)}[j]$ has sensitivity bounded by Δ_f, and the local randomizer is

$$\mathcal{R}\Big(\bar{g}_i^{(2)}[j]\Big) = \bar{g}_i^{(2)}[j] + \xi_i[j], \quad \xi_i[j] \sim \mathrm{Lap}(0, \Delta_f/\epsilon_0);$$

(2) For each coordinate j, each user sends exactly one randomized report $g_i^{(2)'}[j]$;

(3) The reports corresponding to the same coordinate j are anonymized via a shuffling mechanism, which applies a uniformly random permutation and removes all user identifiers.

In addition, the shuffled reports do not contain user-identifying or cross-coordinate linkable metadata that allows the AS to associate reports across different coordinate groups with a specific user. Then, for any $\delta_j \in (0, 1)$, the coordinate-level shuffled mechanism $\mathcal{M}_j = \mathcal{S}_j \circ (g_1^{(2)'}[j], g_2^{(2)'}[j], \ldots, g_n^{(2)'}[j])$ satisfies (ϵ_j, δ_j)-DP with

$$\epsilon_j = O\left((1 \wedge \epsilon_0)e^{\epsilon_0}\sqrt{\frac{\log(1/\delta_j)}{n}}\right).$$

Proof. For a fixed coordinate j, each user U_i applies the local randomizer $\mathcal{R}$ to the j-th coordinate of its clipped less important gradient subvector $\bar{g}_i^{(2)}$, obtaining the randomized report $g_i'^{(2)}[j]$.

For any two possible clipped coordinate values x and x' satisfying $|x - x'| \leq \Delta_f$, the probability density of the Laplace mechanism satisfies

$$\frac{p_{\mathcal{R}(x)}(y)}{p_{\mathcal{R}(x')}(y)} = \frac{\exp\Big(-\epsilon_0|y - x|/\Delta_f\Big)}{\exp\Big(-\epsilon_0|y - x'|/\Delta_f\Big)} \leq \exp\left(\frac{\epsilon_0|x - x'|}{\Delta_f}\right) \leq e^{\epsilon_0}.$$

Therefore, the coordinate-wise Laplace randomizer $\mathcal{R}$ satisfies $(\epsilon_0, 0)$-LDP. The n randomized reports corresponding to coordinate j are then anonymized by a shuffling mechanism $\mathcal{S}_j$, which removes the association between users and their reports. Thus, the coordinate-level mechanism $\mathcal{M}_j$ can be viewed as an instance of the shuffle model consisting of a local randomizer followed by anonymization. By standard privacy amplification results via shuffling [31], the shuffled mechanism satisfies (ϵ_j, δ_j)-DP with

$$\epsilon_j = O\left((1 \wedge \epsilon_0)e^{\epsilon_0}\sqrt{\frac{\log(1/\delta_j)}{n}}\right).$$

□

Theorem 2. *Based on Theorem 1, for all $j \in \{1, \ldots, d_2\}$, the shuffled mechanism $\mathcal{M}_j$ shares identical privacy parameters due to the homogeneous design of the local randomizer and shuffling process, i.e., $(\epsilon_j, \delta_j) = (\epsilon_s, \delta_s)$. Let ϵ_0 denote the coordinate-level local privacy budget assigned to each coordinate-wise randomizer. According to Theorem 1, ϵ_s satisfies*

$$\epsilon_s = O\left((1 \wedge \epsilon_0)e^{\epsilon_0}\sqrt{\frac{\log(1/\delta_s)}{n}}\right).$$

Then, for any $\delta' > 0$, the mechanism $\mathcal{M}_{\text{low}} = (\mathcal{M}_1, \mathcal{M}_2, \ldots, \mathcal{M}_{d_2})$ for the entire less important gradient subvector satisfies $(\epsilon_{\text{low}}, \delta_{\text{low}})$-DP, where

$$\epsilon_{\text{low}} = \sqrt{2d_2\log(1/\delta')}\,\epsilon_s + d_2\epsilon_s\frac{e^{\epsilon_s}-1}{e^{\epsilon_s}+1}, \quad \delta_{\text{low}} = d_2\delta_s + \delta'.$$

Moreover,

$$\epsilon_{\text{low}} \le \sqrt{2d_2\log(1/\delta')}\,\epsilon_s + \frac{d_2}{2}\epsilon_s^2.$$

Proof. By Theorem 1, each coordinate-level shuffled mechanism $\mathcal{M}_j$ satisfies (ϵ_s, δ_s)-DP, where

$$\epsilon_s = O\left((1 \wedge \epsilon_0)e^{\epsilon_0}\sqrt{\frac{\log(1/\delta_s)}{n}}\right).$$

Since $g_i^{(2)}$ contains d_2 coordinates, changing the less important gradient subvector of one user may affect that user's report in all coordinates simultaneously. Therefore, although shuffled privacy amplification is established separately for each coordinate-level mechanism, the privacy guarantee for the entire subvector must be derived via composition over the d_2 coordinates. Applying the advanced composition theorem [45,46] for (ϵ, δ)-differential privacy, for any $\delta' > 0$, the composed mechanism satisfies

$$\epsilon_{\text{low}} = \sqrt{2\log(1/\delta')\sum_{j=1}^{d_2}\epsilon_s^2} + \sum_{j=1}^{d_2}\epsilon_s\frac{e^{\epsilon_s}-1}{e^{\epsilon_s}+1}, \quad \delta_{\text{low}} = \sum_{j=1}^{d_2}\delta_s + \delta'.$$

Since all coordinates share the same privacy parameters, this reduces to

$$\epsilon_{\text{low}} = \sqrt{2d_2\log(1/\delta')}\,\epsilon_s + d_2\epsilon_s\frac{e^{\epsilon_s}-1}{e^{\epsilon_s}+1}, \quad \delta_{\text{low}} = d_2\delta_s + \delta'.$$

Furthermore, since

$$\frac{e^{\epsilon_s}-1}{e^{\epsilon_s}+1} = \tanh\left(\frac{\epsilon_s}{2}\right) \le \frac{\epsilon_s}{2},$$

we obtain

$$\epsilon_{\text{low}} \le \sqrt{2d_2\log(1/\delta')}\,\epsilon_s + \frac{d_2}{2}\epsilon_s^2.$$

Therefore, the composition of the d_2 coordinate-level shuffled mechanisms provides the stated $(\epsilon_{\text{low}}, \delta_{\text{low}})$-DP guarantee for the entire less important gradient subvector. □

Theorem 2, together with Theorem 1, characterizes the privacy amplification effect of coordinate-wise shuffling at the subvector level. Specifically, shuffling converts the coordinate-level local privacy parameter ϵ_0 into an amplified parameter ϵ_s that decreases with the number of shuffled users n, while advanced composition extends these coordinate-level guarantees to the entire less important gradient subvector. Since n corresponds to the number of users in each resulting cluster, the effective shuffle-amplified privacy guarantee may vary across clusters. In particular, a smaller cluster yields a larger ϵ_s and, consequently, a weaker amplified privacy guarantee for the corresponding less important gradient subvector. This cluster-size dependence represents a limitation of the current privacy amplification mechanism, since users in smaller clusters receive weaker shuffle-based amplification. Nevertheless, this variation affects only the additional amplification provided by shuffling. Since the

coordinate-wise Laplace randomization is performed locally before shuffling, the underlying $(\epsilon_0, 0)$-LDP guarantee remains unchanged regardless of the cluster size. No universal minimum cluster size is specified, since the cluster size required to achieve a particular amplified privacy level depends on the target privacy parameters and the local privacy budget ϵ_0. Future work will investigate adaptive minimum-size constraints or fallback mechanisms for small clusters when stronger shuffle-based amplification is required.

Proposition 1. *Under the considered non-collusion assumption, the protocol view of the MS does not provide sufficient information to directly recover a user's complete original local gradient.*

Proof. For user U_i, the MS observes $\text{View}_{\text{MS}}(U_i) = \{\pi_i, g_i^{(2)'}\}$. The permutation sequence π_i provides the structural information required for similarity measurement and clustering, including the ordering information of important gradient coordinates, but it does not reveal their corresponding numerical gradient values. In particular, the MS does not receive the important gradient subvector $\tilde{g}_i^{(1)}$.

For the remaining part, the MS observes only the locally randomized less important gradient subvector $g_i^{(2)'} = \mathcal{R}(\bar{g}_i^{(2)})$, rather than the original $g_i^{(2)}$. Since each coordinate is independently randomized through the Laplace mechanism before transmission, the MS observes the less important gradient components only in randomized form. Therefore, the MS lacks the numerical values of the important gradient components and does not directly observe the original less important gradient components. Consequently, its protocol view does not directly reveal the complete original local gradient g_i. □

Proposition 2. *Under the non-collusion assumption between the MS and AS, the protocol view of the AS does not provide sufficient information to directly recover a user's complete original local gradient.*

Proof. For user U_i, the AS observes the permuted important gradient information $\tilde{g}_i^{(1)}$ together with the anonymously shuffled less important gradient information $\tilde{g}_i^{(2)}$.

For the important part, although the AS obtains the numerical values in $\tilde{g}_i^{(1)}$, it does not know their original coordinate mapping in g_i. Specifically, this mapping is determined by the global permutation sequence π^g, which is held by the MS and is unavailable to the AS under the non-collusion assumption. Hence, the AS cannot directly map the observed important gradient values back to their original coordinates.

For the less important part, each user first applies the coordinate-wise Laplace randomizer and the resulting reports are subsequently anonymized by shuffling. By Theorem 2, the resulting less important gradient mechanism satisfies the derived $(\epsilon_{\text{low}}, \delta_{\text{low}})$ shuffled differential privacy guarantee. This guarantee bounds the AS's ability to distinguish neighboring less important gradient reports; it does not imply that each individual gradient value is deterministically unrecoverable.

Therefore, under the non-collusion assumption, the AS lacks the original coordinate mapping of the important components and receives the less important components only through the randomized shuffled mechanism. Its protocol view thus does not directly reveal the complete original local gradient g_i. □

Proposition 3. *Consider a target user U_i that does not collude with the other $N-1$ users. Collusion between these $N-1$ users and either the MS or the AS does not directly reveal the complete original local gradient of U_i.*

Proof. We consider the two possible collusion cases separately.

Case 1: Collusion with the MS. The colluding users may disclose their own permutation sequences, randomized reports, and other locally known information to the MS. However, the important gradient values of the non-colluding user U_i remain unavailable to the MS, because the MS receives only the permutation information associated with U_i and does not receive $\tilde{g}_i^{(1)}$. Moreover, the less important gradient subvector of U_i is independently randomized by the Laplace mechanism before transmission. Therefore, even after incorporating the information disclosed by the other $N-1$ users, the MS does not directly obtain the complete original gradient of U_i.

Case 2: Collusion with the AS. The colluding users may disclose the global permutation sequence π^g to the AS. In this case, the coordinate-obfuscation protection of the important gradient subvector may be removed, allowing the AS to map $\tilde{g}_i^{(1)}$ back to its original coordinates. Therefore, we do not claim that the important gradient part remains hidden under this collusion scenario.

In addition, when the other $N-1$ users disclose their own randomized reports, the anonymity provided by shuffling can be substantially weakened. After identifying the colluding users' reports, the remaining randomized report may be attributable to U_i. Accordingly, the full shuffle-amplified guarantee of Theorem 2 should not be interpreted as remaining unchanged under this extreme collusion setting.

Nevertheless, the less important report of U_i is still generated by the coordinate-wise Laplace mechanism using randomness independently sampled by U_i. The colluding users and the AS do not obtain this private randomness and therefore observe only the randomized less important report rather than the original $g_i^{(2)}$. Thus, even though the coordinate mapping of the important part may become known, the complete original local gradient of U_i is not directly revealed.

Hence, in both collusion cases, the adversary's combined protocol view does not directly disclose the complete original gradient of the remaining non-colluding user. □

Remark 1. *The formal differential privacy guarantee of PA-CFL applies to the less important gradient subvector. The permutation-based protection of important gradient components is intended to hide their original coordinate mapping from a non-colluding AS, rather than to provide differential privacy for these components. Accordingly, PA-CFL does not claim zero information leakage from the permutation sequences; instead, they expose the structural information required for clustering while withholding the corresponding important gradient values from the MS. Moreover, under the extreme $N-1$-user collusion setting, the anonymity gain provided by shuffling may be reduced, and the residual protection of the less important gradient subvector should be understood as stemming from the target user's local Laplace randomization. Resistance to practical gradient-inversion or data-reconstruction attacks is therefore evaluated empirically rather than claimed as a formal consequence of the above propositions.*

6.2. Model Deviation Analysis

The following assumptions are standard analytical conditions commonly adopted in federated learning theory [36,47] and are introduced to derive tractable theoretical bounds. We note that the convexity assumption restricts the scope of the theoretical convergence analysis and does not strictly characterize the non-convex CNN and ResNet-10 models used in our experiments. Therefore, the theoretical results hold under the stated assumptions, while the practical performance of PA-CFL with non-convex models such as CNN and ResNet-10 is evaluated empirically in Section 7. Following these studies, we first analyze the convergence of our proposed framework under intra-cluster learning and then extend it to the case with global aggregation.

Assumption 1. *For any user $U_i \in U$ belonging to cluster C_y, the local loss function $\mathcal{L}_{i,C_y}(\theta)$ is convex with respect to the model parameter vector $\theta \in \mathbb{R}^d$.*

Assumption 2. *The loss function $\mathcal{L}_{i,C_y}(\theta)$ is β-Lipschitz continuous. That is, for any $\theta_1,\ \theta_2 \in \mathbb{R}^d$, there exist: $\|\mathcal{L}_{i,C_y}(\theta_1) - \mathcal{L}_{i,C_y}(\theta_2)\| \leq \beta\|\theta_1 - \theta_2\|$.*

Assumption 3. *The loss function $\mathcal{L}_{i,C_y}(\theta)$ is $L_{\mathcal{Z}}$-Lipschitz smooth continuous. For any θ_1, $\theta_2 \in \mathbb{R}^d$, there exists: $\|\nabla\mathcal{L}_{i,C_y}(\theta_1) - \nabla\mathcal{L}_{i,C_y}(\theta_2)\| \leq L_{\mathcal{Z}}\|\theta_1 - \theta_2\|$.*

Definition 3. *(Cluster Loss Function). For any cluster $C_y \in C$, where $y \in [0, |C|]$, the cluster loss function is $\mathcal{L}_{C_y}(\cdot) \triangleq \sum_{i\in C_y} w_i\mathcal{L}_{i,C_y}(\cdot)$, where $w_i \geq 0$ and $\sum_{i\in C_y} w_i = 1$.*

Definition 4. *(Intra-cluster Gradient Divergence). For any user $U_i \in U$, the divergence between its loss function and the cluster loss is defined as $\zeta_{i,C_y} \triangleq max_w\|\nabla\mathcal{L}_{i,C_y}(\theta) - \nabla\mathcal{L}_{C_y}(\theta)\|$. And the intra-cluster divergence is defined as $\zeta_{C_y} \triangleq \sum_{i\in C_y} w_i\zeta_{i,C_y}$, where $w_i \geq 0$ and $\sum_{i\in C_y} w_i = 1$.*

Lemma 1. *According to Assumptions 1–3, the cluster loss function $\mathcal{L}_{C_y}(\cdot)$ for each cluster C_y is convex, β-Lipschitz continuous, and $L_{\mathcal{Z}}$-Lipschitz smooth.*

Proof. Based Definition 3, $\mathcal{L}_{C_y}(\theta) = \sum_{U_i\in C_y} w_i\mathcal{L}_{i,C_y}(\theta)$. Thus, we have

- Convexity: Since each $\mathcal{L}_{i,C_y}(\theta)$ is convex and $w_i \geq 0$, $\sum_{i\in C_y} w_i = 1$, their weighted sum $\mathcal{L}_{C_y}(\theta)$ is also convex.
- β-Lipschitz continuous: For any $\theta_1,\ \theta_2$, we have

$$\begin{aligned}\|\mathcal{L}_{C_y}(\theta_1) - \mathcal{L}_{C_y}(\theta_2)\| &= \|\sum_{i\in C_y} w_i\mathcal{L}_{i,C_y}(\theta_1) - \sum_{i\in C_y} w_i\mathcal{L}_{i,C_y}(\theta_2)\| \\ &\leq \sum_{i\in C_y} w_i\beta\|\theta_1 - \theta_2\| = \beta\|\theta_1 - \theta_2\|.\end{aligned}$$

- $L_{\mathcal{Z}}$-Lipschitz smooth: This can be proven similarly to the β-Lipschitz continuity case.

□

Definition 5. *Model Updates. In PA-CFL, the local model update of user $U_i \in C_y$ during round t and local step r is defined as*

$$\theta_{t,r}^{i,C_y} = \begin{cases} \theta_{t,r-1}^{i,C_y} - \eta\nabla\mathcal{L}_{i,C_y}\left(\theta_{t,r-1}^{i,C_y}\right), & r \in [1, R], \\ \theta_t^{C_y} \triangleq \sum_{i\in C_y} w_i\theta_{t-1,R}^{i,C_y}, & r = 0. \end{cases}$$

Similarly, the virtual centralized model of cluster C_y evolves as

$$\hat{\theta}_{t,e}^{C_y} = \hat{\theta}_{t,e-1}^{C_y} - \eta\nabla\mathcal{L}_{C_y}(\hat{\theta}_{t,e-1}^{C_y}), r \in [1, R], \quad \hat{\theta}_{t,e}^{C_y} = \theta_t^{C_y}, r = 0.$$

The case $r = 0$ corresponds to the synchronization step, in which PA-CFL aggregates the final local updates $\theta_{t,R}^{i,C_y}$ across all users in cluster C_y.

Lemma 2. *Let $\hat{\theta}_{t,r}^{C_y}$ denote the virtual model obtained by centralized training using all data in cluster C_y and $\theta_{t,r}^{i,C_y}$ denote the model trained locally by user $U_i \in C_y$. Under Assumptions 1–3, for communication round t, local training step $r \in [0, R]$, and learning rate η, the divergence is bounded by $\|\theta_{t,r}^{i,C_y} - \hat{\theta}_{t,r}^{C_y}\| \leq \frac{\zeta_{i,C_y}}{L_{\mathcal{Z}}}\left((\eta L_{\mathcal{Z}} + 1)^r - 1\right)$.*

Proof.

$$\begin{aligned}
&\|\theta_{t,r}^{i,C_y} - \hat{\theta}_{t,r}^{C_y}\| = \|\theta_{t,r-1}^{i,C_y} - \eta\nabla\mathcal{L}_{i,C_y}\left(\theta_{t,r-1}^{i,C_y}\right) - (\hat{\theta}_{t,r-1}^{C_y} - \eta\nabla\mathcal{L}_{C_y}\left(\hat{\theta}_{t,r-1}^{C_y}\right)\| \\
&\leq \|\theta_{t,r-1}^{i,C_y} - \hat{\theta}_{t,r-1}^{C_y}\| + \eta\|\nabla\mathcal{L}_{i,C_y}(\theta_{t,r-1}^{i,C_y}) - \nabla\mathcal{L}_{i,C_y}(\hat{\theta}_{t,r-1}^{i,C_y})\| \\
&+ \eta\|\nabla\mathcal{L}_{i,C_y}(\hat{\theta}_{t,r-1}^{i,C_y}) - \nabla\mathcal{L}_{C_y}(\hat{\theta}_{t,r-1}^{i,C_y})\| \\
&\leq (\eta L_{\mathcal{Z}} + 1)\|\theta_{t,r-1}^{i,C_y} - \hat{\theta}_{t,r-1}^{C_y}\| + \eta\zeta_{i,C_y}
\end{aligned}$$

Let $\alpha(r) = \|\theta_{t,r-1}^{i,C_y} - \hat{\theta}_{t,r-1}^{C_y}\|$, we have

$$\begin{aligned}
&\alpha(r) \leq (\eta L_{\mathcal{Z}} + 1)\alpha(r-1) + \eta\zeta_{i,C_y} \\
&\Rightarrow \alpha(r) + \frac{\zeta_{i,C_y}}{L_{\mathcal{Z}}} \leq (\eta L_{\mathcal{Z}} + 1)^r\alpha(0) + (\eta L_{\mathcal{Z}} + 1)^r\frac{\zeta_{i,C_y}}{L_{\mathcal{Z}}} \overset{\text{(i)}}{\Rightarrow} \alpha(r) + \frac{\zeta_{i,C_y}}{L_{\mathcal{Z}}} \leq (\eta L_{\mathcal{Z}} + 1)^r\frac{\zeta_{i,C_y}}{L_{\mathcal{Z}}} \\
&\Rightarrow \|\theta_{t,r}^{i,C_y} - \hat{\theta}_{t,r}^{C_y}\| \leq \frac{\zeta_{i,C_y}}{L_{\mathcal{Z}}}((\eta L_{\mathcal{Z}} + 1)^r - 1).
\end{aligned}$$

where (i) is obtained using the recursive relation and using $\alpha(0) = \|\theta_{t,0}^{i,C_y} - \hat{\theta}_{t,0}^{C_y}\| = 0$. □

Lemma 3. *Under Assumptions 1–3, the divergence between the aggregated cluster model* $\theta_t^{C_y} \triangleq \sum_{i\in C_y} w_i\theta_{t,R}^{i,C_y}$ *and the centralized virtual cluster model* $\hat{\theta}_t^{C_y} \triangleq \sum_{i\in C_y} w_i\hat{\theta}_{t,r-1}^{C_y}$ *is upper-bounded as follows:*

$$\|\theta_t^{C_y} - \hat{\theta}_{t,r}^{C_y}\| \leq \frac{\zeta_{C_y}}{L_{\mathcal{Z}}}\left((1 + \eta L_{\mathcal{Z}})^R - 1\right),$$

where $\zeta_{C_y} = \sum_{i\in C_y} w_i\zeta_{i,C_y}$ *denotes the weighted divergence level of users within cluster* C_y.

Proof. From Definition 5, the update rule of the centralized virtual cluster model is $\hat{\theta}_{t,r}^{C_y} = \theta_{t,r-1}^{C_y} - \eta\nabla\mathcal{L}_{C_y}(\theta_{t,r-1}^{C_y}), r \in [1, R]$ and at the aggregation step $r = 0$, $\theta_t^{C_y} = \sum_{i\in C_y} w_i\theta_{t-1,R}^{C_y}$. According to Lemma 2, for any user $U_i \in C_y$, and any local step r, we have $\|\theta_{t,r}^{i,C_y} - \hat{\theta}_{t,r}^{C_y}\| \leq \frac{\zeta_{i,C_y}}{L_{\mathcal{Z}}}\left((\eta L_{\mathcal{Z}} + 1)^R - 1\right)$.

Using this bound, we analyze the deviation between the aggregated cluster model and the centralized model:

$$\begin{aligned}
\|\theta_t^{C_y} - \hat{\theta}_t^{C_y}\| &= \|\sum_{i\in C_y} w_i\left(\theta_{t-1,R}^{i,C_y} - \hat{\theta}_{t-1,R}^{C_y}\right)\| \leq \sum_{i\in C_y} w_i\|\theta_{t-1,R}^{i,C_y} - \hat{\theta}_{t-1,R}^{C_y}\| \\
&\overset{\text{(ii)}}{\leq} \frac{(1 + \eta L_{\mathcal{Z}})^R - 1}{L_{\mathcal{Z}}}\sum_{i\in C_y} w_i\zeta_{i,C_y} \\
&\overset{\text{(iii)}}{=} \frac{\zeta_{C_y}}{L_{\mathcal{Z}}}((1 + \eta L_{\mathcal{Z}})^R - 1),
\end{aligned}$$

where (ii) follows directly from Lemma 2 and (iii) follows Definition 4, where $\zeta_{C_y} = \sum_{i\in C_y} w_i\zeta_{i,C_y}$. Thus, the stated bound holds. □

Definition 6. *Model Updates with Differential Privacy Noise. For analytical tractability, we focus on the effect of differential privacy noise on model updates and abstract away the clipping-induced bias and the cross-user coordinate reassignment introduced by intra-cluster shuffling. Accordingly, we define the iterator of* $\tilde{\theta}_{t,r}^{i,C_y}$ *as follows:*

$$\tilde{\theta}_{t,r}^{i,C_y} = \tilde{\theta}_{t,r-1}^{i,C_y} - \eta\nabla\mathcal{L}_{i,C_y}(\tilde{\theta}_{t,r-1}^{i,C_y}), r \in [1, R]$$

when $r = 0$, $\tilde{\theta}_{t,r}^{i,C_y}$ *is defined as follows:*

$$\tilde{\theta}_t^{C_y} \triangleq \sum_{i \in C_y} w_i \left(\tilde{\theta}_{t-1,R-1}^{i,C_y} - \eta \left(\nabla \mathcal{L}_{i,C_y}(\tilde{\theta}_{t-1,R-1}^{i,C_y}) + n_i \right) \right) = \sum_{i \in C_y} w_i \left(\tilde{\theta}_{t-1,R}^{i,C_y} - \eta n_i \right).$$

For the convergence analysis, let $n_i \in \mathbb{R}^{d_2}$ *denote the Laplace noise added to the less important gradient subvector of user* U_i*, where its coordinates are independently sampled as* $n_i[j] \sim \mathrm{Lap}\left(0, \frac{\Delta_f}{\epsilon_0}\right), j \in \{1, \ldots, d_2\}$*. Since a Laplace random variable with scale parameter* b *has variance* $2b^2$*, we have* $\mathbb{E}[n_i] = 0$*, and* $\mathbb{E}[\|n_i\|^2] = 2d_2\left(\frac{\Delta_f}{\epsilon_0}\right)^2$*. We assume that the noise vectors are independently sampled across usersand communication rounds.*

Theorem 3. *Suppose Assumptions 1–3 hold. Based on Definitions 3–6 and combining Lemmas 1–3, under the analytical abstraction introduced in Definition 6, we consider a fixed cluster* C_y *and condition on its aggregation weights* $\{w_i\}_{i \in C_y}$*, which are treated as fixed with respect to the independently sampled differential privacy noise. Define* $q = (1 + \eta L_{\mathcal{Z}})^R$*,* $H_{C_y} = \frac{\zeta_{C_y}}{L_{\mathcal{Z}}}(q-1)$*, and* $V_{C_y} = 2d_2\left(\frac{\Delta_f}{\epsilon_0}\right)^2 \sum_{i \in C_y} w_i^2$*. If the privacy-preserving cluster model and the centralized virtual model are initialized from the same model, then their expected squared model deviation after* t *communication rounds is bounded by*

$$\mathbb{E}\left[\left\| \tilde{\theta}_t^{C_y} - \hat{\theta}_t^{C_y} \right\|^2 \right] \leq \left(\frac{q^t - 1}{q - 1} \right)^2 \left(H_{C_y} + \eta \sqrt{V_{C_y}} \right)^2, \quad q \neq 1.$$

This result characterizes the finite-round deviation between the privacy-preserving cluster model and its centralized virtual counterpart, rather than asserting asymptotic convergence to an optimal solution.

Proof. Step 1: $r \in [1, R]$. From Definitions 5 and 6, the privacy-preserving local update $\tilde{\theta}_{t,r}^{i,C_y}$ and the centralized virtual model $\hat{\theta}_{t,r}^{C_y}$ satisfy $\tilde{\theta}_{t,r}^{i,C_y} = \tilde{\theta}_{t,r-1}^{i,C_y} - \eta \nabla \mathcal{L}_{i,C_y}(\tilde{\theta}_{t,r-1}^{i,C_y})$ and $\hat{\theta}_{t,r}^{C_y} = \hat{\theta}_{t,r-1}^{C_y} - \eta \nabla \mathcal{L}_{C_y}(\hat{\theta}_{t,r-1}^{C_y})$. Thus,

$$\begin{aligned} \left\| \tilde{\theta}_{t,r}^{i,C_y} - \hat{\theta}_{t,r}^{C_y} \right\| &\leq \left\| \tilde{\theta}_{t,r-1}^{i,C_y} - \hat{\theta}_{t,r-1}^{C_y} \right\| + \eta \left\| \nabla \mathcal{L}_{i,C_y}(\tilde{\theta}_{t,r-1}^{i,C_y}) - \nabla \mathcal{L}_{i,C_y}(\hat{\theta}_{t,r-1}^{C_y}) \right\| \\ &\quad + \eta \left\| \nabla \mathcal{L}_{i,C_y}(\hat{\theta}_{t,r-1}^{C_y}) - \nabla \mathcal{L}_{C_y}(\hat{\theta}_{t,r-1}^{C_y}) \right\| \\ &\leq (1 + \eta L_{\mathcal{Z}}) \left\| \tilde{\theta}_{t,r-1}^{i,C_y} - \hat{\theta}_{t,r-1}^{C_y} \right\| + \eta \zeta_{i,C_y}. \end{aligned}$$

Applying the same recursion as in Lemma 2 gives

$$\left\| \tilde{\theta}_{t,r}^{i,C_y} - \hat{\theta}_{t,r}^{C_y} \right\| \leq (1 + \eta L_{\mathcal{Z}})^r \left\| \tilde{\theta}_{t,0}^{i,C_y} - \hat{\theta}_{t,0}^{C_y} \right\| + \frac{\zeta_{i,C_y}}{L_{\mathcal{Z}}} ((1 + \eta L_{\mathcal{Z}})^r - 1).$$

Step 2: $r = 0$. Let $D_t = \left(\mathbb{E}[\|\tilde{\theta}_t^{C_y} - \hat{\theta}_t^{C_y}\|^2] \right)^{1/2}$. After the R local steps, aggregation of the randomized less important gradient components introduces the weighted noise $\bar{n}_t = \sum_{i \in C_y} w_i n_i$. Under the fixed-weight condition stated in the theorem, the aggregation weights $\{w_i\}_{i \in C_y}$ are treated as fixed with respect to the independently sampled differential privacy noise. Accordingly, all expectations below are taken over the differential privacy noise conditional on these fixed weights.

Using the bound obtained in Step 1, together with $q = (1 + \eta L_{\mathcal{Z}})^R$ and $\zeta_{C_y} = \sum_{i \in C_y} w_i \zeta_{i,C_y}$, we obtain

$$\left\| \tilde{\theta}_t^{C_y} - \hat{\theta}_t^{C_y} \right\| \leq q \left\| \tilde{\theta}_{t-1}^{C_y} - \hat{\theta}_{t-1}^{C_y} \right\| + \frac{\zeta_{C_y}}{L_{\mathcal{Z}}}(q-1) + \eta \|\bar{n}_t\| = q \left\| \tilde{\theta}_{t-1}^{C_y} - \hat{\theta}_{t-1}^{C_y} \right\| + H_{C_y} + \eta \|\bar{n}_t\|.$$

Since the weights are fixed and the noise vectors are independently sampled across users with zero mean, the cross terms vanish. Therefore,

$$\mathbb{E}\Big[\|\bar{n}_t\|^2\Big] = \mathbb{E}\left[\left\|\sum_{i\in C_y} w_i n_i\right\|^2\right] = \sum_{i\in C_y} w_i^2 \mathbb{E}\Big[\|n_i\|^2\Big] = 2d_2\left(\frac{\Delta_f}{\epsilon_0}\right)^2 \sum_{i\in C_y} w_i^2 = V_{C_y}.$$

Applying the Minkowski inequality in L_2 to the above deviation bound gives $D_t \le qD_{t-1} + H_{C_y} + \eta\sqrt{V_{C_y}}$. Since the privacy-preserving cluster model and the centralized virtual model are initialized identically, $D_0 = 0$. Solving the recursion yields

$$D_t \le \Big(H_{C_y} + \eta\sqrt{V_{C_y}}\Big)\sum_{\tau=0}^{t-1} q^\tau = \frac{q^t-1}{q-1}\Big(H_{C_y} + \eta\sqrt{V_{C_y}}\Big), \qquad q \neq 1.$$

Squaring both sides gives

$$\mathbb{E}\left[\left\|\widetilde{\theta}_t^{C_y} - \hat{\theta}_t^{C_y}\right\|^2\right] \le \left(\frac{q^t-1}{q-1}\right)^2\Big(H_{C_y} + \eta\sqrt{V_{C_y}}\Big)^2.$$

Substituting $q = (1+\eta L_{\mathcal{Z}})^R$, $H_{C_y} = \frac{\zeta_{C_y}}{L_{\mathcal{Z}}}(q-1)$, and $V_{C_y} = 2d_2\left(\frac{\Delta_f}{\epsilon_0}\right)^2 \sum_{i\in C_y} w_i^2$ completes the proof. □

We next extend the above finite-round model-deviation result to the global aggregation rounds, i.e., the rounds satisfying $t \bmod \kappa = 0$.

Let $\bar{\theta}_t^{C_y}$ denote the model parameter of cluster C_y after global aggregation. According to the global aggregation rule of PA-CFL, the AS first computes the normalized global gradient

$$G_t = \sum_{C_l \in C_{\text{final}}} \frac{|C_l|}{\sum_{C_j\in C_{\text{final}}}|C_j|} \frac{g_t^{C_l}}{\|g_t^{C_l}\|},$$

where $g_t^{C_l}$ denotes the intra-cluster aggregated gradient of cluster C_l at round t.

For cluster C_y, let $\Gamma_t^{C_y} \triangleq \|G_t - g_t^{C_y}\|$ denote the discrepancy between the global gradient and the intra-cluster gradient that would otherwise be used by cluster C_y. Accordingly, the globally synchronized model can be related to the intra-cluster model as $\bar{\theta}_t^{C_y} = \widetilde{\theta}_t^{C_y} - \eta(G_t - g_t^{C_y})$. Therefore, by the triangle inequality, $\|\bar{\theta}_t^{C_y} - \hat{\theta}_t^{C_y}\| \le \|\widetilde{\theta}_t^{C_y} - \hat{\theta}_t^{C_y}\| + \eta\Gamma_t^{C_y}$. Applying the Minkowski inequality in L_2 and using the preceding theorem, the finite-round model-deviation bound after global aggregation is

$$\mathbb{E}\left[\left\|\bar{\theta}_t^{C_y} - \hat{\theta}_t^{C_y}\right\|^2\right] \le \left[\frac{q^t-1}{q-1}\Big(H_{C_y} + \eta\sqrt{V_{C_y}}\Big) + \eta\Big(\mathbb{E}\Big[(\Gamma_t^{C_y})^2\Big]\Big)^{1/2}\right]^2,$$

where $q = (1+\eta L_{\mathcal{Z}})^R$, $H_{C_y} = \frac{\zeta_{C_y}}{L_{\mathcal{Z}}}(q-1)$, and $V_{C_y} = 2d_2\left(\frac{\Delta_f}{\epsilon_0}\right)^2 \sum_{i\in C_y} w_i^2$.

For rounds satisfying $t \bmod \kappa \neq 0$, no global synchronization is performed, and the intra-cluster model-deviation bound established in the preceding theorem applies directly.

7. Experimental Evaluation

7.1. Experimental Setup

We implement a prototype of PA-CFL using the PyTorch framework (version 2.7.0.dev20250127, CUDA 12.6). In this setup, each user possesses a test set that follows the same data distribution as their training set. All experiments were conducted on a CPU (i5-14600KF) and a GPU (RTX

4080S). Following recent federated learning studies [48–50] we evaluate PA-CFL on three widely used datasets:

- MNIST [51] is a widely used dataset of handwritten digits, consisting of 70,000 grayscale images with pixel dimensions of 28×28.
- Fashion-MNIST (FMNIST) [52] is a dataset of grayscale images depicting 10 categories of fashion items. It contains 70,000 images, each sized 28×28 pixels, with labels indicating the item category.
- CIFAR-10 [53] is a benchmark dataset composed of 60,000 color images of size 32×32, categorized into 10 object classes such as animals, vehicles, and everyday items.

For MNIST and FashionMNIST, we use a standard CNN with two 5×5 conv layers, each followed by ReLU and 2×2 max-pooling, and two fully connected layers. For CIFAR-10, we adopt ResNet-10, a lightweight residual network with four residual stages and a final fully connected layer. To assess PA-CFL under heterogeneous data, we consider two representative Non-IID settings reflecting real-world heterogeneity:

- Non-IID Set-1 (Dirichlet label skew): We adopt the Dirichlet distribution to generate heterogeneous label distributions across users. Specifically, for each dataset, the samples of all classes are partitioned among users according to a Dirichlet distribution with concentration parameter α. We set $\alpha \in \{0.1, 0.5, 1.0\}$, where a smaller α indicates a higher degree of statistical heterogeneity.
- Non-IID Set-2 (Clustered label partition): Users are evenly divided into five clusters, with each cluster being assigned samples from a specific subset of classes. For notational convenience, we use label indices $\{0, \ldots, 9\}$ to represent the ten classes in each dataset. The class partitions for the five clusters are defined as $\{0,1\}$, $\{2,3\}$, $\{4,5\}$, $\{6,7\}$, and $\{8,9\}$, respectively.

All experiments are implemented based on the federated learning framework proposed in [54]. To reduce the computational overhead of pairwise structural similarity computation, FastDTW [37] is adopted throughout the experiments to approximate the DTW distance between user permutation sequences. All experiments are repeated five times with different random seeds, and the average results are reported.

Unless otherwise stated, PA-CFL follows the complete pipeline described in Section 5. We use top-k permutation sequences constructed from the top-k gradient elements and the global importance score for structural representation, FastDTW with MST–DBI for pre-clustering, and differentiated privacy protection based on permutation obfuscation, Laplace randomization, and intra-cluster shuffling. We set $k = 0.0025d$ and $\epsilon_0 = 1$, with the clipping threshold determined by the median L_2 norm. The AS subsequently performs cosine-distance-based fine clustering, softmax-weighted cluster aggregation, and periodic global aggregation.

7.2. Accuracy Comparison

7.2.1. Baseline

We compare PA-CFL with three categories of representative methods:

- Conventional FL: FedAvg [1], FedProx [9], MOON [12], and FedLC [13], which improve optimization without explicit clustering.
- Clustered FL: CFL [10], APCFL [16], ProCFL [55], FedDAG [5], and EBS-CFL [6] which address data heterogeneity by partitioning users into clusters and training cluster-specific models.
- Privacy-preserving aggregation: ShieldFL [24], FedPHE [25], and Split Aggregation [56], which focus on secure aggregation under privacy constraints without explicitly modeling user clusters.

7.2.2. Result Analysis

We evaluate PA-CFL and the baseline methods under different heterogeneous data environments. The number of local epochs is set to $R = 3$, and the total number of users is $N = 20$. According to the two Non-IID settings described earlier, four experimental scenarios are considered. For MNIST and Fashion-MNIST, we adopt a CNN model with learning rate $\eta = 0.01$, batch size 64, and 100 communication rounds, while for CIFAR-10, we use ResNet-10 with learning rate $\eta = 0.005$, batch size 256, and 200 communication rounds. For a fair comparison, we also includes a non-private variant of PA-CFL, denoted as PA-CFL*, since some baselines either do not provide privacy protection or adopt lossless privacy-preserving mechanisms.

As shown in Figure 5, PA-CFL achieves faster and more stable convergence than all baselines, while consistently attaining higher accuracy across all datasets under privacy protection. In general, clustering-based FL methods, including CFL, APCFL, ProCFL, PA-CFL, and PA-CFL*, outperform conventional methods such as FedAvg, FedProx, MOON, FedPHE, Split Aggregation, and ShieldFL, as clustering mitigates statistical heterogeneity by enabling users with similar data distributions to train more specialized models. Accordingly, most non-clustering baselines suffer clearer degradation under Non-IID Set 2, where the highly skewed data distribution further exposes the limitation of a single global model. As heterogeneity increases, the performance of non-clustering methods further degrades due to larger discrepancies among local updates, leading to more severe aggregation errors. In contrast, clustering-based methods generally benefit from stronger heterogeneity, highlighting the advantage of personalization. Among them, PA-CFL* achieves the best overall performance, indicating that the superiority of our framework stems not only from clustering itself but also from producing more accurate user partitions. With the proposed adaptive clustering strategy and structure-aware similarity metric, PA-CFL* effectively groups users with similar data distributions, reducing intra-cluster inconsistency and improving cluster-wise training. Although PA-CFL is slightly inferior to PA-CFL* due to privacy perturbation, it still consistently outperforms all privacy-preserving baselines, demonstrating that the proposed privacy mechanism preserves most of the clustering utility under heterogeneous data.

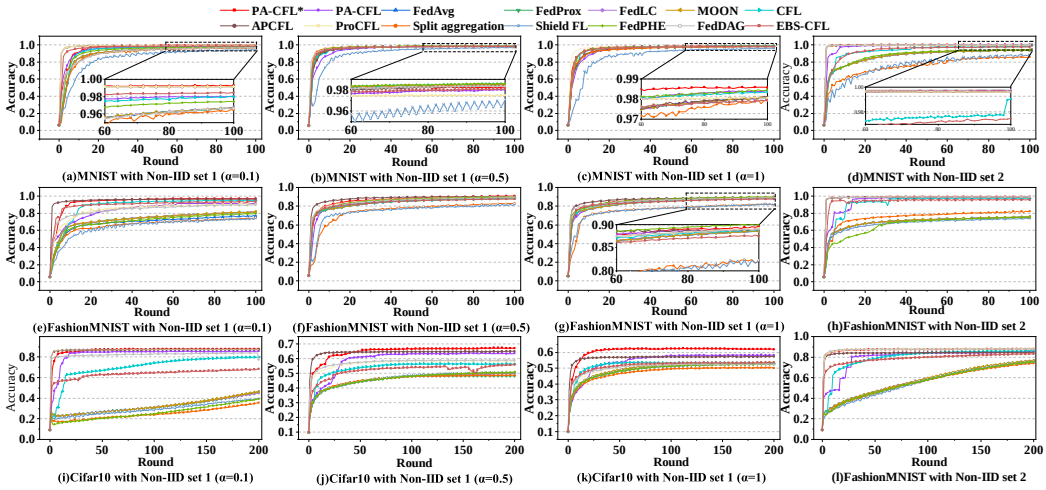


Figure 5. Performance comparison of accuracy with state-of-the-art baselines in different Non-IID data distributions over MNIST, F-MNIST, and CIFAR10.

Regarding individual baselines, CFL achieves relatively high accuracy in some cases, but converges more slowly due to its hierarchical clustering process. APCFL generally outperforms CFL and ProCFL by constraining local updates toward cluster models, while

ProCFL performs well under Non-IID Set 2 by leveraging label-distribution similarity. FedDAG also achieves competitive performance across several settings, particularly on FashionMNIST and some CIFAR-10 settings. EBS-CFL performs well on MNIST and FashionMNIST, but exhibits a more noticeable performance degradation under highly heterogeneous CIFAR-10 settings. Nevertheless, PA-CFL and PA-CFL* still achieve better convergence and higher final accuracy across different data distributions, indicating that our clustering strategy yields higher-quality cluster structures.

7.3. Efficiency Evaluation

To evaluate the efficiency of the proposed PA-CFL scheme in the privacy-preserving phase, we compare it with several representative privacy-preserving federated learning methods, including ShieldFL [24], FedPHE [25], and Split Aggregation [56]. Specifically, we focus on the computational and communication overhead introduced by the privacy-preserving mechanisms, rather than the overall training process.

Table 1 reports the actual communication overhead (in MB) and computational overhead (in seconds) under the same experimental settings as the accuracy evaluation, providing concrete efficiency measurements for different datasets and model architectures. In the following subsections, we further evaluate the communication and computational overhead under varying numbers of users and gradient dimensions. In addition, we provide a detailed breakdown of the computational overhead introduced by different components of PA-CFL.

Table 1. Model parameters, communication, and computation overhead of different schemes.

Scheme	Dataset	Model	Params.	Comm. (MB)	Comp. (s)
PA-CFL	MNIST/FashionMNIST	CNN	582,026	884.84	12.43
Split Aggregation			582,026	932.58	36.52
FedPHE			582,026	4882.38	63.98
ShieldFL			582,026	5329.03	685,580
PA-CFL	CIFAR-10	ResNet10	4,910,922	746.60	10.49
Split Aggregation			4,910,922	786.88	30.82
FedPHE			4,910,922	4119.58	53.98
ShieldFL			4,910,922	4496.44	1.88×10^6

7.3.1. Computational Overhead

Figure 6a,b present the computational overhead of different methods. ShieldFL incurs the highest computational overhead because it relies on homomorphic encryption, which involves expensive modular exponentiation operations. In contrast, PA-CFL, FedPHE, and Split Aggregation exhibit significantly lower computational overhead. Specifically, FedPHE reduces the encryption overhead via packed homomorphic encryption, while Split Aggregation employs lightweight masking operations. In PA-CFL, permutation is applied only to the important gradient subvector, while the less important gradient subvector is protected by the lightweight local randomization and shuffling. Therefore, PA-CFL achieves the lowest computational overhead among the compared methods.

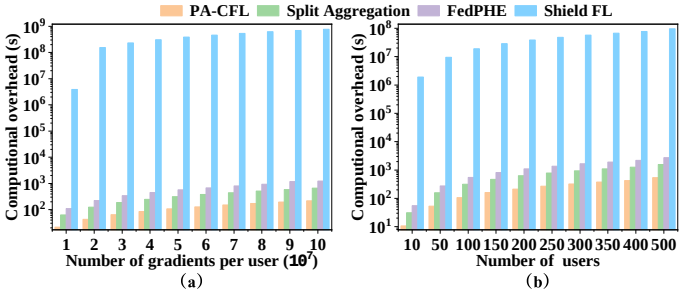


Figure 6. Computational overhead per user in the privacy-preserving phase. (**a**) Varying the number of gradient dimensions with $N = 20$. (**b**) Varying the number of users with $d = 10^7$.

In addition to the privacy-preserving phase, we further evaluate the computational overhead introduced by the clustering procedure. Figure 7 reports the computation time under varying model dimensions and numbers of users. Specifically, we vary the model dimension from 10^7 to 10^8 with $N = 20$, and increase the number of users from 10 to 500 with 10^7. For PA-CFL, we separately measure the overhead of pairwise DTW distance computation and MST construction in the pre-clustering stage. We also measure the computational overhead of cosine-distance computation and clustering in the fine clustering stage.

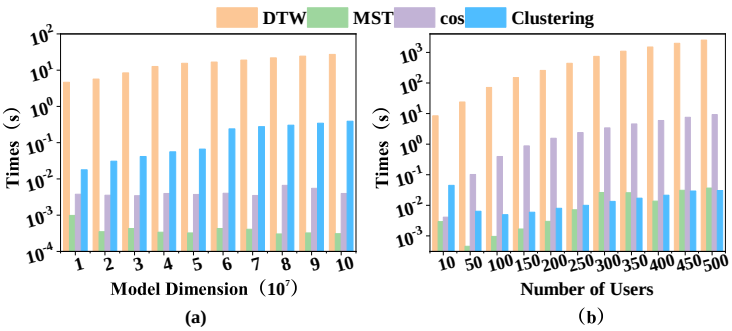


Figure 7. Computational overhead of PA-CFL's procedure under varying model dimensions and numbers of users. (**a**) Varying the model dimension with $N = 20$. (**b**) Varying the number of users with $d = 10^7$.

As the model dimension or the number of users increases, DTW distance computation becomes the dominant computational cost, whereas MST construction introduces only negligible additional overhead. In particular, the increase with the number of users is mainly caused by the$N(N-1)/2$ pairwise distance computations. These results demonstrate the computational behavior of the DTW-based pre-clustering procedure under increasing model dimensions and user scales, while also showing that pairwise DTW computation becomes the primary scalability bottleneck as the number of users grows.

7.3.2. Communication Overhead

Figure 8a,b report the communication overhead of different methods. ShieldFL introduces the highest communication overhead because homomorphic encryption enlarges the ciphertext size. Although FedPHE also adopts homomorphic encryption, it improves efficiency by packing multiple model parameters into a single ciphertext. Split Aggregation requires each user to upload two masked model updates to two servers, which also increases the communication overhead. In PA-CFL, each user uploads an important gradient subvector and a less important gradient subvector obtained via permutation and randomization, respectively. As these two subvectors together have nearly the same size as the original gradient, the communication overhead remains low. Overall, PA-CFL incur the lowest communication overhead among the compared methods.

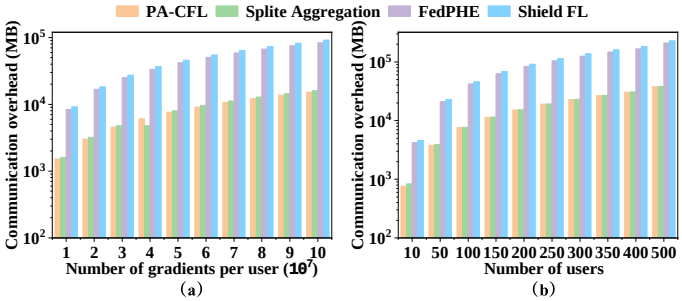


Figure 8. Communication overhead per user in the privacy-preserving phase. (**a**) Varying the number of gradient dimensions with $N = 20$. (**b**) Varying the number of users with $d = 10^7$.

7.4. Effectiveness Analysis

7.4.1. Trade-Off Between Privacy and Utility

We investigate the impact of the coordinate-level local privacy budget ϵ_0 and top-k ratio k on both model utility and privacy leakage. The learning rate is set to 0.005, the number of local epochs is $R = 3$, the number of communication rounds is 50, and the batch size is 64. Before local randomization, the less important gradient subvector is clipped in the L_2 norm using the median norm following prior work [57]. Laplace noise sampled from $\mathrm{Lap}(0, \Delta_f/\epsilon_0)$ is then added independently to each coordinate, where ϵ_0 denotes the local privacy budget assigned to each coordinate-wise randomizer.

As shown in Figure 9a, the training loss increases as the coordinate-level local privacy budget ϵ_0 decreases, since a smaller ϵ_0 requires stronger noise perturbation during training. Figure 9b shows that increasing k generally reduces the training loss, because fewer gradient elements are perturbed by noise, while the elements in the less important subvector usually have relatively small magnitudes and limited impact on optimization.

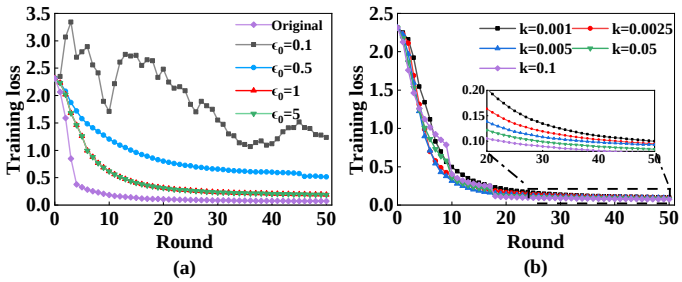


Figure 9. Training FashionMNIST (Non-IID Set-2). (**a**) Varying coordinate-level local privacy budgets ϵ_0. (**b**) Varying top-k.

To further evaluate the privacy impact of k, we adopt the gradient inversion attack Deep Leakage from Gradients (DLG) [58], which reconstructs training data from shared gradients. In PA-CFL, users upload permutation information corresponding to important gradient coordinates, while coordinate obfuscation is introduced through permutation operations on the important gradient subvectors.

Figure 10 (a) shows the reconstruction results using only the top-k gradient elements. As k increases from $0.1\% \times d$ to $10\% \times d$, the reconstructed images become progressively clearer, especially when $k \geq 5\% \times d$. Figure 10 (b) presents the reconstruction results at the AS using shuffled top-k elements. Compared with Figure 10 (a), the reconstructed images become significantly less distinguishable, demonstrating the effectiveness of the proposed privacy-preserving mechanism. However, larger k values still lead to clearer reconstructions. Figure 10 (c) shows the reconstruction results at the MS using only the uploaded permutation sequences, where the reconstructed images are highly indistinguishable, since coordinate indices alone do not provide the gradient magnitude information required for accurate reconstruction.

Based on these observations and considering both utility and privacy, we empirically set the coordinate-level local privacy budget $\epsilon_0 = 1$ and $k = 0.0025d$ in the subsequent experiments. The values $\epsilon_0 = 1$ and $k = 0.0025d$ are empirical defaults for the models evaluated in this work and are not universally optimal. Since k denotes the number of retained gradient coordinates and is set proportionally to the model dimension d, it scales with model size. However, the optimal retention ratio may still vary across datasets and model architectures. For larger models such as ResNet-50, $k = 0.0025d$ can be used as an initial setting and adjusted through validation.

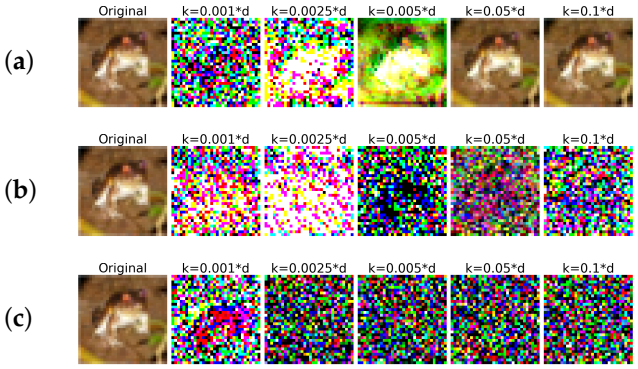


Figure 10. Comparison of gradient reconstruction results under different exposure settings. (a) Reconstruction using only the top-k gradient elements; (b) reconstruction at the AS using shuffled top-k elements; (c) reconstruction at the MS using only the uploaded permutation sequences.

7.4.2. The Effectiveness of Our Proposed Structure-Aware Similarity Metric

To evaluate the effectiveness of our proposed DTW-based structure-aware similarity metric, we conduct experiments on the MNIST dataset with seven users $\{U_1, U_2, \ldots, U_7\}$. User U_1 holds labels 0–4, while user U_2 possesses labels 5–9, resulting in no overlap. The remaining users are constructed with increasing label overlap ratios of 0%, 20%, 40%, 60%, 80%, and 100% with respect to U_1. We compute the distance between each user's model update and that of user U_1 using several commonly used metrics, including Euclidean (L2) distance, Manhattan (L1) distance, cosine distance, Pearson distance, MaxNorm distance, and Mahalanobis distance. All distance values are normalized by the maximum value of each metric to ensure comparability across different metrics.

Figure 11a shows that as the overlap ratio increases, our metric exhibits the steepest decline and smoothest trend, indicating higher sensitivity to data distribution differences. In the similarity range of 0–0.8, Mahalanobis distance, MaxNorm distance, L1, and L2 remain significantly higher than cosine distance, Pearson distance, and our metric, suggesting these methods struggle to distinguish users with similar distributions.

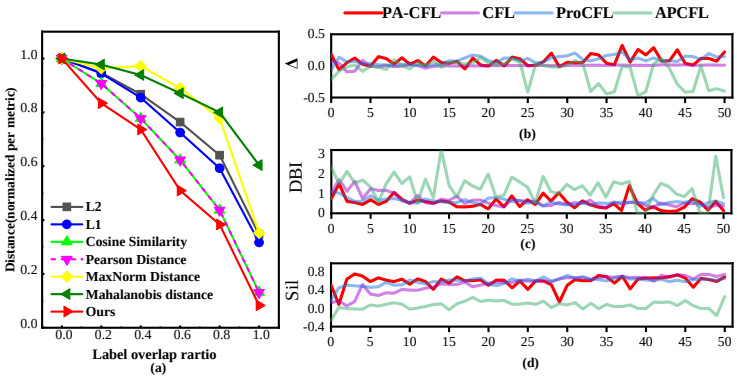


Figure 11. Comparison of similarity metrics and adaptive clustering performance on MNIST. (**a**) Distance values (normalized per metric) under different label overlap ratios. (**b**) Δ, (**c**) Davies–Bouldin Index (DBI), and (**d**) silhouette coefficient (Sil). Settings: $\eta = 0.01$, $R = 3$, batch size 64, CNN model.

7.4.3. Effectiveness of the Adaptive Clustering Scheme

To evaluate the effectiveness of our proposed adaptive clustering scheme, we conduct experiments on the MNIST dataset under Non-IID setting 2 for 50 communication rounds. We compare PA-CFL with several representative clustered federated learning methods, including CFL, ProCFL, and APCFL. To assess clustering quality, we report three clustering-related metrics over training rounds: Davies–Bouldin Index (DBI), silhouette coefficient (Sil) [59], and the difference (Δ) between the minimum inter-cluster distance and the maximum intra-cluster distance.

As observed from Figure 11 b–d, PA-CFL achieves lower DBI values and higher Sil values than the compared methods in most training rounds, indicating more compact intra-cluster structures and better inter-cluster separation. Meanwhile, PA-CFL maintains a relatively stable and larger Δ, suggesting clearer separation between clusters. In contrast, APCFL exhibits significant fluctuations. while CFL and ProCFL show relatively stable trends but consistently lower Δ values, indicating inferior cluster separation. These results demonstrate that the proposed adaptive clustering scheme more effectively captures underlying data-distribution similarities among users and maintains a more reliable clustering structure under heterogeneous settings.

7.5. Ablation Study

In Table 2, we conduct an ablation study to evaluate the contribution of each component in PA-CFL, including the privacy-preserving module (PB), the pre-clustering module (PC), and the fine-clustering module (FC). When both PC and FC are removed, the cluster structure required by PB is no longer available. Treating all users as a single cluster would effectively introduce a different global privacy-preserving setting rather than constituting a direct ablation of PA-CFL.

We first investigate the effect of the pre-clustering module (PC), which captures user similarity through the proposed structure-aware metric and provides a coarse initialization for subsequent refinement. As shown in Table 2, removing PC leads to noticeable performance degradation across all three datasets when PB and FC are retained. Specifically, without PC, the method reduces to a simpler clustering strategy without structure-aware similarity modeling, leading to accuracy decreases of 0.48–4.80% on MNIST, 0.41–5.79% on Fashion-MNIST, and 1.49–2.25% on CIFAR-10. This indicates that PC is crucial for separating users with heterogeneous data distributions and improving clustering quality.

Next, we study the effect of the fine-clustering module (FC), which further refines the coarse grouping based on user-side gradient information. To isolate this effect, we compare variants with and without FC while keeping PC enabled and PB disabled. Removing FC

again leads to consistent performance drops across all datasets, with decreases of 0.31–2.57% on MNIST, 0.13–1.54% on Fashion-MNIST, and 0.78–15.03% on CIFAR-10. The larger drop on CIFAR-10, particularly under stronger heterogeneity, suggests that coarse clustering alone is insufficient, while FC enables more precise cluster refinement.

We further examine the variant with PB and PC enabled but FC removed. Compared with the full PA-CFL, removing FC results in accuracy decreases of 0.66–5.66% on MNIST, 0.88–8.27% on Fashion-MNIST, and 3.47–7.44% on CIFAR-10. This result further shows that, although PC can provide an effective coarse partition under privacy protection, the resulting clusters still require subsequent refinement. In particular, the more evident degradation on Fashion-MNIST and CIFAR-10 indicates that FC plays an important role in correcting imperfect coarse assignments under more complex and heterogeneous data distributions.

Finally, we analyze the impact of the privacy-preserving module (PB). Comparing the full PA-CFL with the counterpart without PB while keeping PC and FC enabled, we observe only moderate accuracy degradation, namely 0.04–1.36% on MNIST, 0.30–7.23% on Fashion-MNIST, and 1.81–4.58% on CIFAR-10. More importantly, when PB is combined with PC, the degradation remains smaller than that of the variant applying privacy protection without PC, indicating that pre-clustered privacy protection is more robust and better alleviates the adverse effect of privacy perturbation.

Table 2. Ablation study of different components (PB, PC, FC) under different Non-IID settings.

PB	PC	FC	MNIST				Fashion-MNIST				CIFAR-10			
			Non-IID Set 1			Non-IID Set 2	Non-IID Set 1			Non-IID Set 2	Non-IID Set 1			Non-IID Set 2
			0.1	0.5	1.0	/	0.1	0.5	1.0	/	0.1	0.5	1.0	/
✓	✓	✓	98.05 ± 0.14	98.41 ± 0.25	98.02 ± 0.27	99.70 ± 0.05	91.64 ± 0.17	88.69 ± 0.27	89.11 ± 0.32	98.57 ± 0.13	85.63 ± 0.09	63.81	58.47	85.67
✓	✗	✓	95.52 ± 0.26	93.61 ± 0.43	96.88 ± 0.41	99.22 ± 0.18	85.85 ± 0.39	85.6 ± 0.40	86.07 ± 0.46	98.16 ± 0.17	83.78 ± 0.38	62.32 ± 0.55	56.25 ± 0.76	83.42 ± 0.12
✓	✓	✗	92.89 ± 0.23	92.75 ± 0.48	95.77 ± 0.39	99.04 ± 0.14	83.37 ± 0.35	83.83 ± 0.42	84.49 ± 0.53	97.69 ± 0.22	82.16 ± 0.40	59.76 ± 0.63	51.03 ± 0.85	80.29 ± 0.17
✗	✓	✓	99.41 ± 0.01	98.84 ± 0.01	98.23 ± 0.02	99.74 ± 0.01	98.87 ± 0.01	91.55 ± 0.01	90.76 ± 0.01	98.87 ± 0.00	87.49 ± 0.01	68.39 ± 0.01	62.51 ± 0.03	87.48 ± 0.01
✗	✓	✗	96.84 ± 0.13	97.40 ± 0.10	98.08 ± 0.11	99.43 ± 0.08	91.89 ± 0.09	91.24 ± 0.12	90.63 ± 0.08	98.74 ± 0.05	72.46 ± 0.16	65.82 ± 0.13	61.73 ± 0.11	74.86 ± 0.07
✗	✗	✓	98.08 ± 0.01	97.33 ± 0.02	98.03 ± 0.02	99.52 ± 0.01	96.43 ± 0.06	89.72 ± 0.04	88.87 ± 0.04	98.59 ± 0.04	86.43 ± 0.04	66.95 ± 0.06	61.23 ± 0.07	85.58 ± 0.04

Setting: $N = 20$ and $R = 3$. Four scenarios are considered under two Non-IID settings. For MNIST and Fashion-MNIST, we use a CNN with $\eta = 0.01$, batch size 64, and 100 rounds; for CIFAR-10, we use ResNet-10 with $\eta = 0.005$, batch size 256, and 200 rounds. ✓ indicates that the corresponding component is enabled, while $\times$ indicates that it is disabled.

8. Conclusions

In this paper, we present PA-CFL, a privacy-preserving adaptive clustering framework for federated learning under heterogeneous data that effectively tackles the challenges posed by data heterogeneity and privacy protection. Unlike prior works, PA-CFL enables efficient clustering of users based on their data distributions without requiring manually specified clustering-related parameters, while improving model accuracy, particularly in highly heterogeneous data environments. PA-CFL integrates a structure-aware similarity metric and an adaptive clustering scheme, together with a lightweight privacy-preserving mechanism, achieving a favorable trade-off between privacy protection and model utility with low computational and communication overhead. Extensive experimental results demonstrate that PA-CFL outperforms existing methods in terms of accuracy under heterogeneous data distributions. Furthermore, the proposed privacy-preserving mechanism achieves better efficiency compared with state-of-the-art approaches.

Despite these advantages, PA-CFL still has limitations that warrant further investigation. The pairwise similarity computation in the pre-clustering stage introduces increasing computational overhead as the number of users grows, which may limit its scalability in large-scale FL systems. Moreover, the current accuracy evaluation is limited to three image benchmarks with $N = 20$ users, and further validation on larger-scale and real-world FL scenarios is needed. Since CFL partitions users into different clusters and performs model aggregation within each cluster, it also provides a promising framework for incorporating Byzantine robustness. Future work will investigate more efficient similarity computation and clustering strategies, extend PA-CFL to more diverse and larger-scale FL scenarios, and explore Byzantine-robust detection and aggregation mechanisms within the adaptive clustering framework.

Author Contributions: Conceptualization, T.Z. and W.S.; methodology, T.Z. and W.S.; software, T.Z.; validation, T.Z. and H.Z.; formal analysis, T.Z. and C.Z.; investigation, T.Z.; writing—original draft preparation, T.Z.; writing—review and editing, W.S.; visualization, T.Z.; supervision, W.S.; project administration, W.S.; funding acquisition, W.S. H.Z. and C.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This research is supported by the Shandong Provincial Natural Science Foundation, China (ZR2025MS1006, ZR2024MF013), Youth Innovation Technology Project of Higher School in Shandong Province (2023KJ365), China Postdoctoral Science Foundation (2025M781443), Scientific Research Foundation for High-level Talents of Anhui University of Science and Technology (2025yjrc0117), Qingdao Science and Technology Demonstration Project (26-1-1-sfgc-2-hy), and Qingdao Key Technology Tackling Program (26-1-1-gjgg-29-gx).

Data Availability Statement: No new data were created or analyzed in this study. Data sharing is not applicable to this article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. McMahan, H.B.; Moore, E.; Ramage, D.; Hampson, S.; y Arcas, B.A. Communication-Efficient Learning of Deep Networks from Decentralized Data. In Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, Fort Lauderdale, FL, USA, 20–22 April 2017; Volume 54, pp. 1273–1282.
2. Zhang, M.; Li, Y.; Wei, F.; Wang, B.; Zhang, Y. Robust clustering federated learning with trusted anchor clients. *J. Inf. Secur. Appl.* **2025**, *94*, 104210.
3. Tang, S.; Wu, B.; Li, Y.; Wang, L.; Yuan, H. Direction-aware local differential privacy for federated learning. *J. Inf. Secur. Appl.* **2026**, *102*, 104562.
4. Farahani, B.; Tabibian, S.; Ebrahimi, H. Toward a personalized clustered federated learning: A speech recognition case study. *IEEE Internet Things J.* **2023**, *10*, 18553–18562.

5. Pramanik, A.; Kantarcioglu, M.; Oria, V.; Sharma, S. FedDAG: Clustered Federated Learning via Global Data and Gradient Integration for Heterogeneous Environments. In Proceedings of the The Fourteenth International Conference on Learning Representations, Rio de Janeiro, Brazil, 23–27 April 2026.
6. Li, Z.; Bao, H.; Guan, M.; Pan, H.; Huang, C.; Dai, H.N. EBS-CFL: Efficient and Byzantine-Robust Secure Clustered Federated Learning. *Proc. AAAI Conf. Artif. Intell.* **2025**, *39*, 18593–18601.
7. Wu, P.; Imbiriba, T.; Closas, P. A Bayesian Framework for Clustered Federated Learning. *IEEE Trans. Pattern Anal. Mach. Intell.* **2026**, *48*, 3471–3481.
8. Kim, H.; Kim, H.; De Veciana, G. Clustered Federated Learning via Gradient-Based Partitioning. In Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria, 21–27 July 2024; Volume 235, pp. 24137–24193.
9. Li, T.; Sahu, A.K.; Talwalkar, A.; Smith, V. Federated learning: Challenges, methods, and future directions. *IEEE Signal Process. Mag.* **2020**, *37*, 50–60.
10. Sattler, F.; Müller, K.R.; Samek, W. Clustered Federated Learning: Model-Agnostic Distributed Multi-Task Optimization Under Privacy Constraints. *IEEE Trans. Neural Netw. Learn. Syst.* **2021**, *32*, 3710–3722.
11. Chen, Z.; Yu, S.; Chen, F.; Wang, F.; Liu, X.; Deng, R.H. Lightweight privacy-preserving cross-cluster federated learning with heterogeneous data. *IEEE Trans. Inf. Forensics Secur.* **2024**, *19*, 7404–7419.
12. Li, Q.; He, B.; Song, D. Model-contrastive federated learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*; IEEE: Piscataway, NJ, USA, 2021; pp. 10713–10722.
13. Zhang, J.; Li, Z.; Li, B.; Xu, J.; Wu, S.; Ding, S.; Wu, C. Federated Learning with Label Distribution Skew via Logits Calibration. In Proceedings of the 39th International Conference on Machine Learning, Baltimore, MD, USA, 17–23 July 2022; Volume 162, pp. 26311–26329.
14. Jaramillo-Civill, M.; Wu, P.; Closas, P. DPMM-CFL: Clustered Federated Learning via Dirichlet Process Mixture Model Nonparametric Clustering. In *Proceedings of the ICASSP 2026—IEEE International Conference on Acoustics, Speech and Signal Processing*; IEEE: Piscataway, NJ, USA, 2026; pp. 1066–1069.
15. Ghosh, A.; Chung, J.; Yin, D.; Ramchandran, K. An Efficient Framework for Clustered Federated Learning. In Proceedings of the Advances in Neural Information Processing Systems, Vancouver, BC, Canada, 6–12 December 2020; Volume 33, pp. 19586–19597.
16. Cao, Y.; Ma, J.; He, Z.; Li, Y. AP-CFL: Clustered federated learning through dynamic clustering and adaptive participation in heterogeneous IoT. *IEEE Internet Things J.* **2025**, *12*, 13671–13682.
17. Guo, Y.; Tang, X.; Lin, T. Enhancing Clustered Federated Learning: Integration of Strategies and Improved Methodologies. In Proceedings of the The Thirteenth International Conference on Learning Representations, Singapore, 24–28 April 2025.
18. Licciardi, A.; Leo, D.; Fanì, E.; Caputo, B.; Ciccone, M. Interaction-Aware Gaussian Weighting for Clustered Federated Learning. In Proceedings of the 42nd International Conference on Machine Learning, Vancouver, BC, Canada, 13–19 July 2025; Volume 267, pp. 37642–37666.
19. Miao, Y.; Liu, Z.; Li, H.; Choo, K.K.R.; Deng, R.H. Privacy-preserving Byzantine-robust federated learning via blockchain systems. *IEEE Trans. Inf. Forensics Secur.* **2022**, *17*, 2848–2861.
20. Wang, T.; Yang, Q.; Zhu, K.; Wang, J.; Su, C.; Sato, K. LDS-FL: Loss Differential Strategy Based Federated Learning for Privacy Preserving. *IEEE Trans. Inf. Forensics Secur.* **2024**, *19*, 1015–1030.
21. Zhao, P.; Cao, Z.; Jiang, J.; Zhang, G. Practical Private Aggregation in Federated Learning Against Inference Attack. *IEEE Internet Things J.* **2023**, *10*, 318–329.
22. Miao, Y.; Yan, X.; Li, X.; Xu, S.; Liu, X.; Li, H.; Deng, R.H. RFed: Robustness-Enhanced Privacy-Preserving Federated Learning Against Poisoning Attack. *IEEE Trans. Inf. Forensics Secur.* **2024**, *19*, 5814–5827.
23. Zhang, C.; Weng, J.; Weng, J.; Zhong, Y.; Liu, J.N.; Deng, C. Robust and Secure Federated Learning with Verifiable Differential Privacy. *IEEE Trans. Dependable Secur. Comput.* **2025**, *22*, 5713–5729.
24. Ma, Z.; Ma, J.; Miao, Y.; Li, Y.; Deng, R.H. ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning. *IEEE Trans. Inf. Forensics Secur.* **2022**, *17*, 1639–1654.
25. Li, Y.; Yan, N.; Chen, J.; Wang, X.; Hong, J.; He, K.; Wang, W.; Li, B. FedPHE: A Secure and Efficient Federated Learning via Packed Homomorphic Encryption. *IEEE Trans. Dependable Secur. Comput.* **2025**, *22*, 5448–5463.
26. Chaudhari, H.; Rachuri, R.; Suresh, A. Trident: Efficient 4PC framework for privacy preserving machine learning. *arXiv* **2019**, arXiv:1912.02631.
27. Sav, S.; Pyrgelis, A.; Troncoso-Pastoriza, J.R.; Froelicher, D.; Bossuat, J.P.; Sousa, J.S.; Hubaux, J.P. POSEIDON: Privacy-Preserving Federated Neural Network Learning. In Proceedings of the 28th Annual Network and Distributed System Security Symposium (NDSS), Virtual, 21–25 February 2021; pp. 1–18.
28. Liu, Z.; Ye, H.; Jiang, Y.; Shen, J.; Guo, J.; Tjuawinata, I.; Lam, K.Y. Privacy-preserving federated unlearning with certified client removal. *IEEE Trans. Inf. Forensics Secur.* **2025**, *20*, 3966–3978.
29. Wang, R.; Zhou, Y.; Liu, J.; Liu, X.; Hu, M.; Wu, D.; Sheng, Q.Z.; Guo, S. CODP: Improving Differentially Private Federated Learning by Cascading and Offsetting Noises Between Iterations. *IEEE Trans. Dependable Secur. Comput.* **2025**, *22*, 5943–5955.

30. Sun, K.; Xu, H.; Hua, K.; Lin, X.; Li, G.; Jiang, T.; Li, J. Joint top-K sparsification and shuffle model for communication-privacy-accuracy tradeoffs in federated-learning-based IoV. *IEEE Internet Things J.* **2024**, *11*, 19721–19735.
31. Balle, B.; Bell, J.; Gascón, A.; Nissim, K. The Privacy Blanket of the Shuffle Model. In *Proceedings of the Advances in Cryptology—CRYPTO 2019*; Springer: Cham, Switzerland, 2019; pp. 638–667.
32. Zhou, Z.; Xu, C.; Wang, M.; Kuang, X.; Zhuang, Y.; Yu, S. A multi-shuffler framework to establish mutual confidence for secure federated learning. *IEEE Trans. Dependable Secur. Comput.* **2022**, *20*, 4230–4244.
33. Xu, H.; Li, J.; Wu, W.; Ren, H. Federated Learning with Sample-level Client Drift Mitigation. In Proceedings of the AAAI Conference on Artificial Intelligence, Philadelphia, PA, USA, 25 February–4 March 2025; Volume 39, pp. 21752–21760.
34. Mukhtiar, N.; Mahmood, A.; Sheng, Q.Z. CoRe-Fed: Bridging Collaborative and Representation Fairness via Federated Embedding Distillation. In Proceedings of the AAAI Conference on Artificial Intelligence, Singapore, 20–27 January 2026; Volume 40, pp. 24458–24466.
35. Guo, K.; Ding, Y.; Liang, J.; Wang, Z.; He, R.; Tan, T. Exploring Vacant Classes in Label-Skewed Federated Learning. In Proceedings of the AAAI Conference on Artificial Intelligence, Philadelphia, PA, USA, 25 February–4 March 2025; Volume 39, pp. 16960–16968.
36. Duan, M.; Liu, D.; Ji, X.; Wu, Y.; Liang, L.; Chen, X.; Tan, Y.; Ren, A. Flexible Clustered Federated Learning for Client-Level Data Distribution Shift. *IEEE Trans. Parallel Distrib. Syst.* **2022**, *33*, 2661–2674.
37. Salvador, S.; Chan, P. Toward accurate dynamic time warping in linear time and space. *Intell. Data Anal.* **2007**, *11*, 561–580.
38. Davies, D.L.; Bouldin, D.W. A cluster separation measure. *IEEE Trans. Pattern Anal. Mach. Intell.* **1979**, *1*, 224–227.
39. Pu, L.; Gu, J.; Lin, C.; Huang, X. Janus: Dual-Server Multi-Round Secure Aggregation with Verifiability for Federated Learning. In Proceedings of the 42nd International Conference on Machine Learning (ICML), Vancouver, BC, Canada, 13–19 July 2025; Volume 267, pp. 49927–49945.
40. Mai, P.; Ding, Y.; Lyu, Z.; Du, M.; Pang, Y. SecEmb: Sparsity-Aware Secure Federated Learning of On-Device Recommender System with Large Embedding. In Proceedings of the 42nd International Conference on Machine Learning (ICML), Vancouver, BC, Canada, 13–19 July 2025; Volume 267, pp. 42642–42667.
41. Shen, X.; Jia, Y.; Xu, C.; Zhu, L. TCFL: Trapdoor Homomorphic Encryption Based Federated Learning Aggregator with Filter. *Peer-to-Peer Netw. Appl.* **2026**, *19*, 58.
42. Zhong, Y.; Ren, Y.; Guo, Z.; Mo, Y. Privacy-preserving personalized federated learning with collusion-resistant verifiability. *J. Inf. Secur. Appl.* **2026**, *100*, 104485.
43. Li, X.; Yang, X.; Zhou, Z.; Lu, R. Efficiently achieving privacy preservation and poisoning attack resistance in federated learning. *IEEE Trans. Inf. Forensics Secur.* **2024**, *19*, 4358–4373.
44. Liu, J.; Li, X.; Liu, X.; Zhang, H.; Miao, Y.; Deng, R.H. DefendFL: A Privacy-Preserving Federated Learning Scheme Against Poisoning Attacks. *IEEE Trans. Neural Netw. Learn. Syst.* **2025**, *36*, 9098–9111.
45. Dwork, C.; Roth, A. The Algorithmic Foundations of Differential Privacy. *Found. Trends Extregistered Theor. Comput. Sci.* **2014**, *9*, 211–407.
46. Kairouz, P.; Oh, S.; Viswanath, P. The Composition Theorem for Differential Privacy. In Proceedings of the 32nd International Conference on Machine Learning, Lille, France, 7–9 July 2015; Volume 37, pp. 1376–1385.
47. Xu, S.; Zheng, Y.; Hua, Z. Harnessing Sparsification in Federated Learning: A Secure, Efficient, and Differentially Private Realization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*; Association for Computing Machinery: New York, NY, USA, 2025; pp. 2354–2368.
48. Wang, X.; Xiong, G.; Cao, H.; Li, J.; Liu, Y. Decentralized Federated Learning with Model Caching on Mobile Agents. In Proceedings of the AAAI Conference on Artificial Intelligence, Philadelphia, PA, USA, 25 February–4 March 2025; Volume 39, pp. 21296–21303.
49. Wang, H.; Gao, W. Tackling Intertwined Data and Device Heterogeneities in Federated Learning with Unlimited Staleness. In Proceedings of the AAAI Conference on Artificial Intelligence, Philadelphia, PA, USA, 25 February–4 March 2025; Volume 39, pp. 21080–21089.
50. Yang, Q.; Chen, L.; Xue, M.; Li, S. Tackling Resource-Constrained and Data-Heterogeneity in Federated Learning with Double-Weight Sparse Pack. In Proceedings of the AAAI Conference on Artificial Intelligence, Singapore, 20–27 January 2026; Volume 40, pp. 27592–27600.
51. Deng, L. The MNIST Database of Handwritten Digit Images for Machine Learning Research. *IEEE Signal Process. Mag.* **2012**, *29*, 141–142.
52. Xiao, H.; Rasul, K.; Vollgraf, R. Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv* **2017**, arXiv:1708.07747.
53. Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images*; Technical Report; University of Toronto: Toronto, ON, Canada, 2009.

54. Zhang, J.; Liu, Y.; Hua, Y.; Wang, H.; Song, T.; Xue, Z.; Ma, R.; Cao, J. PFLlib: A Beginner-Friendly and Comprehensive Personalized Federated Learning Library and Benchmark. *J. Mach. Learn. Res.* **2025**, *26*, 1–10.
55. Xu, Y.; Tan, Y.; Zhang, C.; Sun, P.; Zhang, Y.; Ren, J.; Jiang, H.; Zhang, Y. Towards Privacy-Enhanced and Robust Clustered Federated Learning. *IEEE Trans. Mob. Comput.* **2025**, *24*, 7171–7188.
56. Lu, Z.; Lu, S.; Cui, Y.; Tang, X.; Wu, J. Split aggregation: Lightweight privacy-preserving federated learning resistant to byzantine attacks. *IEEE Trans. Inf. Forensics Secur.* **2024**, *19*, 5575–5590.
57. Andrew, G.; Thakkar, O.; McMahan, H.B.; Ramaswamy, S. Differentially Private Learning with Adaptive Clipping. In *Proceedings of the Advances in Neural Information Processing Systems*; Curran Associates Inc.: Red Hook, NY, USA, 2021; Volume 34, pp. 17455–17466.
58. Zhu, L.; Liu, Z.; Han, S. Deep Leakage from Gradients. In *Proceedings of the Advances in Neural Information Processing Systems*; Curran Associates Inc.: Red Hook, NY, USA, 2019; Volume 32.
59. Licciardi, A.; Leo, D.; Fanì, E.; Caputo, B.; Ciccone, M. Interaction-Aware Gaussian Weighting for Clustered Federated Learning. In Proceedings of the 42nd International Conference on Machine Learning, Vancouver, BC, Canada, 13–19 July 2025; Volume 267, pp. 37642–37666.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.